WaveFEM

DOCUMENTATION

User Manual

How to build, run and analyse electromagnetic simulations in WaveFEM, from importing a model to exporting results.

CONTENTS

About WaveFEM 1
Installation 2
The Interface 4
How WaveFEM Works 9
Workflow 10
Importing Models 11
Materials 15
Ports 17
Simulation Settings 23
Running Simulations 26
Results 27
Design Variants 32
AI Agents 33
Horn Antenna 35
Microstrip Filter 37
Scripted Waveguide Bend 39
References 42

Screenshots are captured from the application by manual/capture.py · Generated 01 November 2026

INTRODUCTION

About WaveFEM

WaveFEM is a three-dimensional electromagnetic simulator based on the finite element method (FEM). It is built for microwave and RF design: filters, waveguide components and antennas. Everything runs from one desktop application, from importing geometry to plotting fields, S-parameters and radiation patterns.

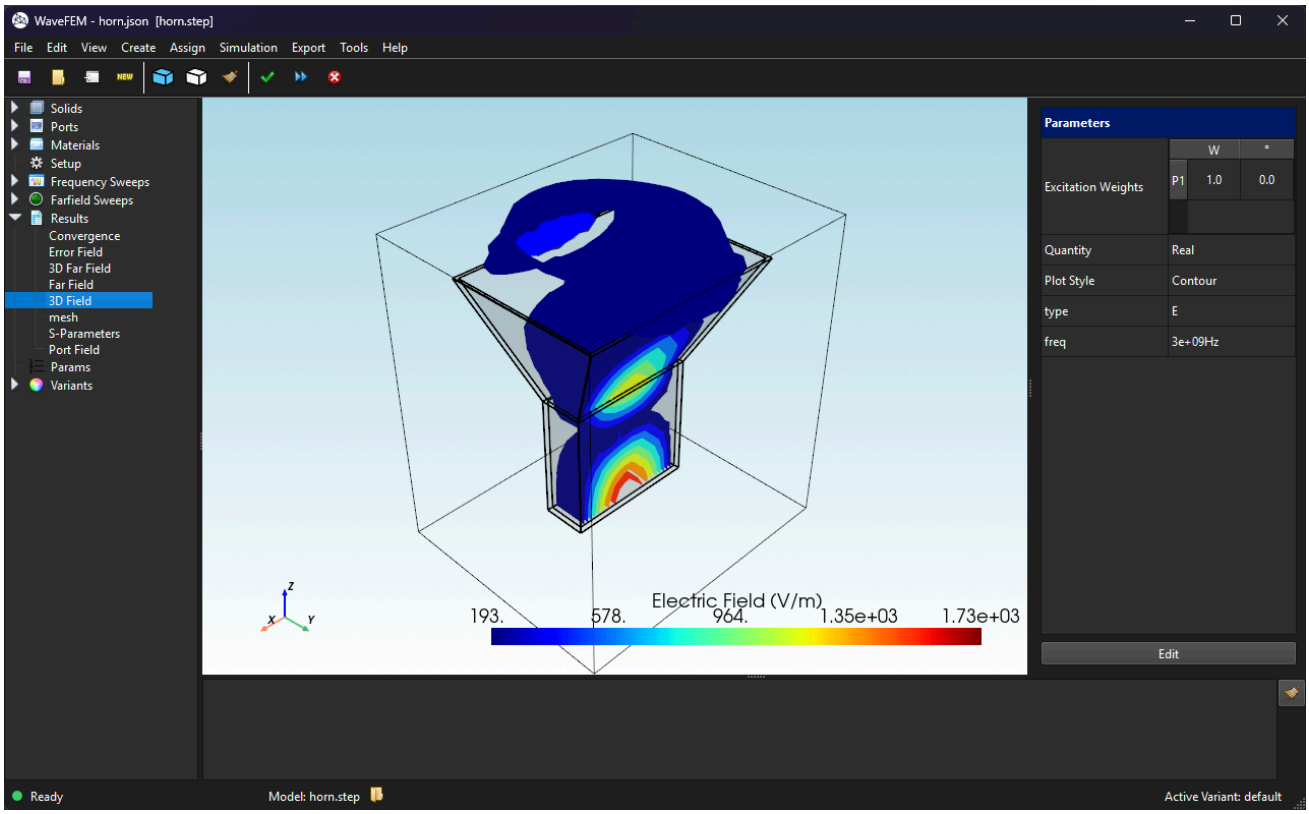


Figure 1 — Electric field inside a horn antenna

This manual covers

- Installing and starting WaveFEM.
- Importing models from STEP, DXF, GDS and Python (CadQuery) scripts.
- Setting up materials, ports, sweeps and solver settings.
- Running simulations and reading the console.
- Viewing, comparing and exporting results.
- Letting an AI agent build and run simulations through the MCP server.
- Worked examples you can follow end to end.

It does not teach electromagnetic theory or derive the solver's equations; see the references for that.

GETTING STARTED

Installation

WaveFEM needs a 64-bit x86 PC running Windows 11. Unzip the download anywhere; no installer is needed. The folder contains:

`assets/`	Icons used by the program.
`core/`	The simulation engine.
`gui/`	The user interface.
`examples/`	Example projects and their model files.
`run.bat`	Starts WaveFEM.

Run **run.bat**. WaveFEM opens with an empty project:

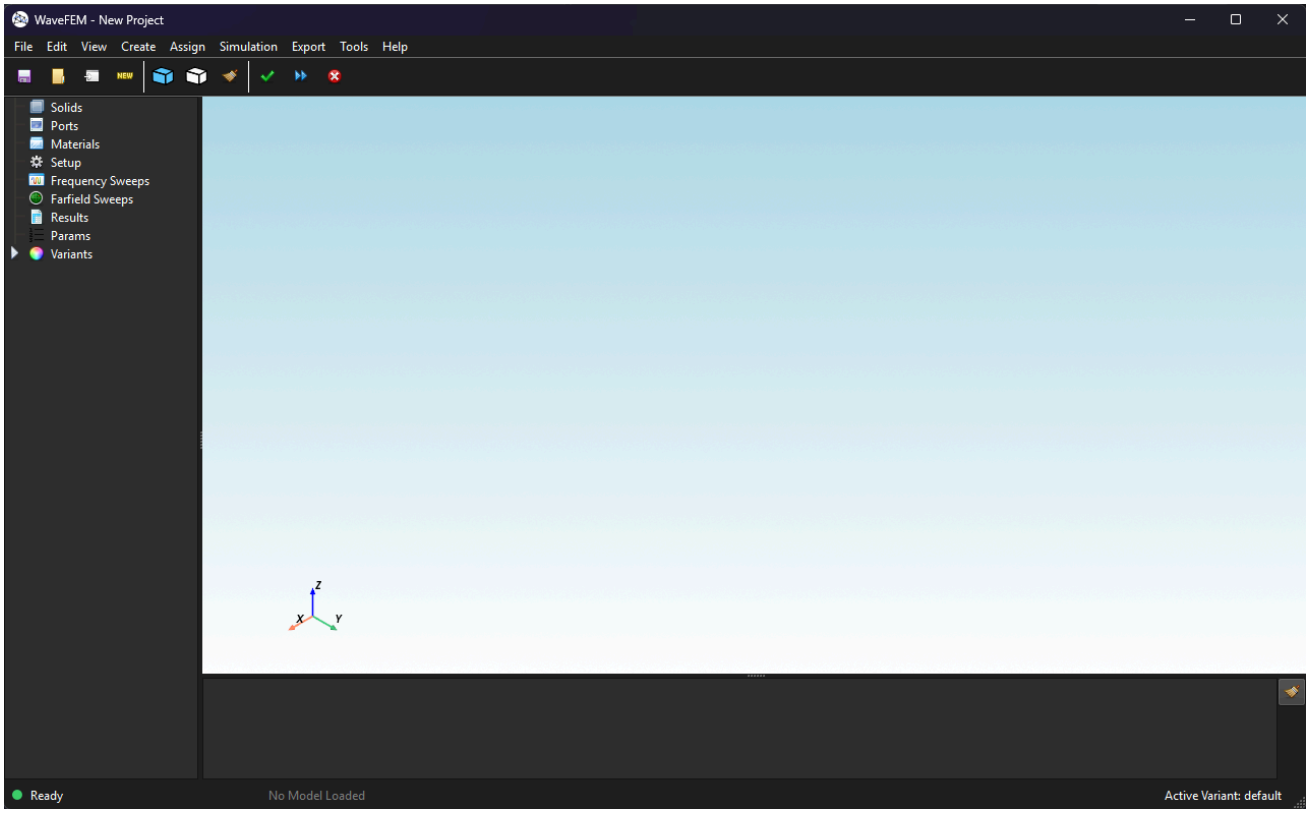


Figure 2 — WaveFEM after start-up

To explore, open an example with **File › Open** and pick a `.json` file in `examples/`. The worked examples walk through complete simulations.

GPU solver (optional)

By default WaveFEM solves on the CPU. On a PC with an NVIDIA graphics card it can instead use the NVIDIA cuDSS solver on the GPU, which is turned on with the `cudss` setting (see Solver settings). The cuDSS libraries ship with WaveFEM in `core/`. You need to provide:

NVIDIA GPU	Turing generation or newer (GeForce RTX 20-series, Quadro RTX, T4 and later). Older cards are not supported by CUDA 13.
NVIDIA driver	A driver that supports CUDA 13, version 580 or newer. Install the latest driver for your card from nvidia.com.

CUDA Toolkit 13.x	Install it from NVIDIA's CUDA downloads page with the default options. WaveFEM finds it automatically. Other CUDA major versions (such as 12.x) do not work.

Restart WaveFEM after installing the toolkit.

If `cudss` is on but the GPU solver cannot start, the run stops with an error in the **Console** that gives the reason:

- *cublas64_13.dll could not be loaded*: the CUDA 13 Toolkit is not installed, or WaveFEM was started before it was installed.
- *cudss64_0.dll could not be loaded*: the file is missing from `core/`. Unzip WaveFEM again.
- *no usable CUDA driver/GPU* or *no CUDA GPU found*: there is no NVIDIA GPU, or the driver is missing or too old for CUDA 13.

Turn `cudss` off to go back to the CPU solver.

GETTING STARTED

The Interface

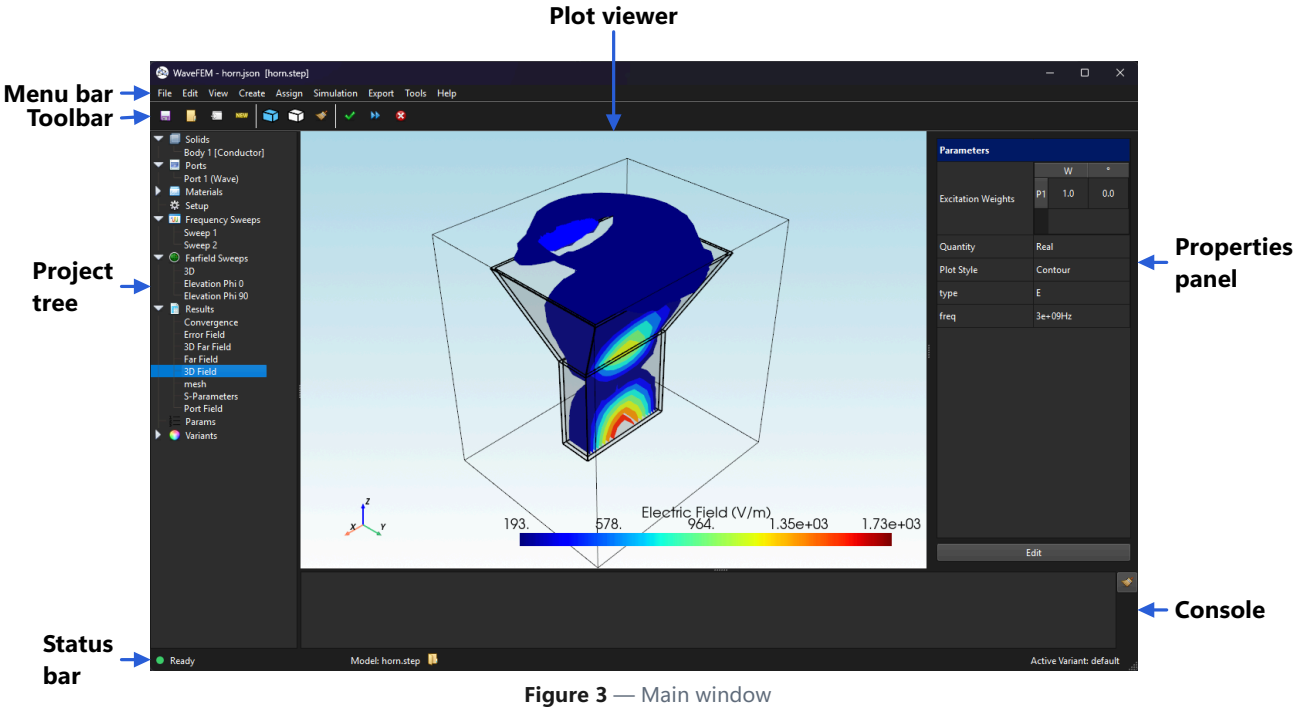
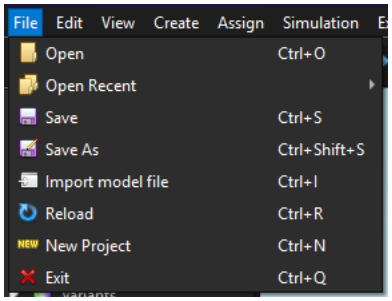


Figure 3 — Main window

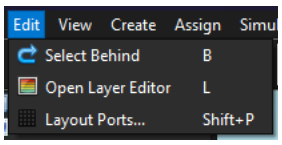
Menu bar	Every command, grouped by task. Most have a keyboard shortcut.
Toolbar	One-click access to the most used commands.
Project tree	Everything in the project: solids, ports, materials, settings, sweeps, results and variants. Click an item to inspect or edit it.
Plot viewer	The 3D model, or the plot of the selected result.
Properties panel	Settings of the selected tree item, with buttons to edit it.
Console	Progress of running simulations, and a record of every error.
Status bar	Loaded model file, busy indicator, short confirmations such as **Saved** and the active design variant.

Menus



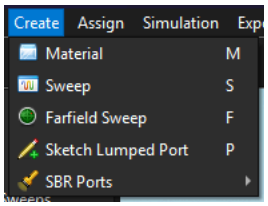
File

Open, save and create projects; import or reload the model file.



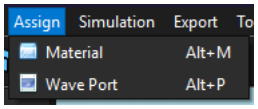
Edit

Select Behind (`B`) cycles to the face behind the selected one; **Open Layer Editor** edits the stack-up of a planar design and **Layout Ports…** draws ports on its layout.



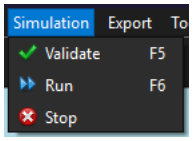
Create

New materials, frequency sweeps, far-field sweeps and lumped ports.



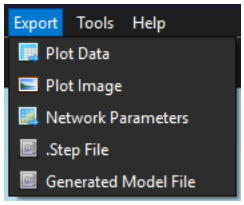
Assign

Turn the selected face into a wave port, or give the selected solids a material.



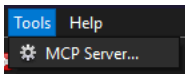
Simulation

Validate the setup, run the simulation, or stop it.



Export

Save the current plot's data or image, Touchstone S-parameters, or the model as a STEP file.



Tools

MCP Server… lets an AI agent work on the open project (see AI Agents).

The **View** menu switches the model between shaded and wireframe display.

Toolbar



Figure 4 — Toolbar buttons

1 Save
2 Open
3 Import model file
4 New Project
5 Shaded View
6 Wireframe View
7 Clear Plot
8 Validate
9 Run Simulation
10 Stop Simulation

The plot viewer has its own toolbar when a 2D plot is shown (zoom, pan and axis settings).

Project tree

Items with an arrow expand to show their contents. Click an item to show its settings in the properties panel or, for results, to plot it. Click empty space in the tree or the viewer to clear the selection.

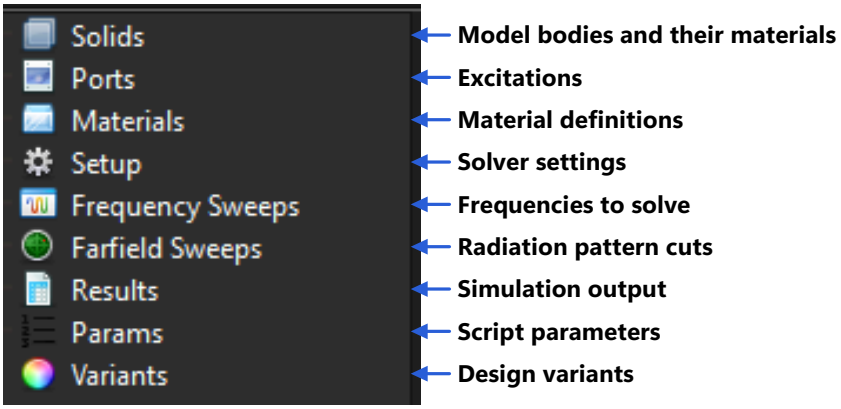


Figure 5 — Project tree

Plot viewer

The viewer shows the 3D model until a result is selected. Drag to rotate, scroll to zoom. Right-click the model to select a face; click a solid in the tree to select a body.

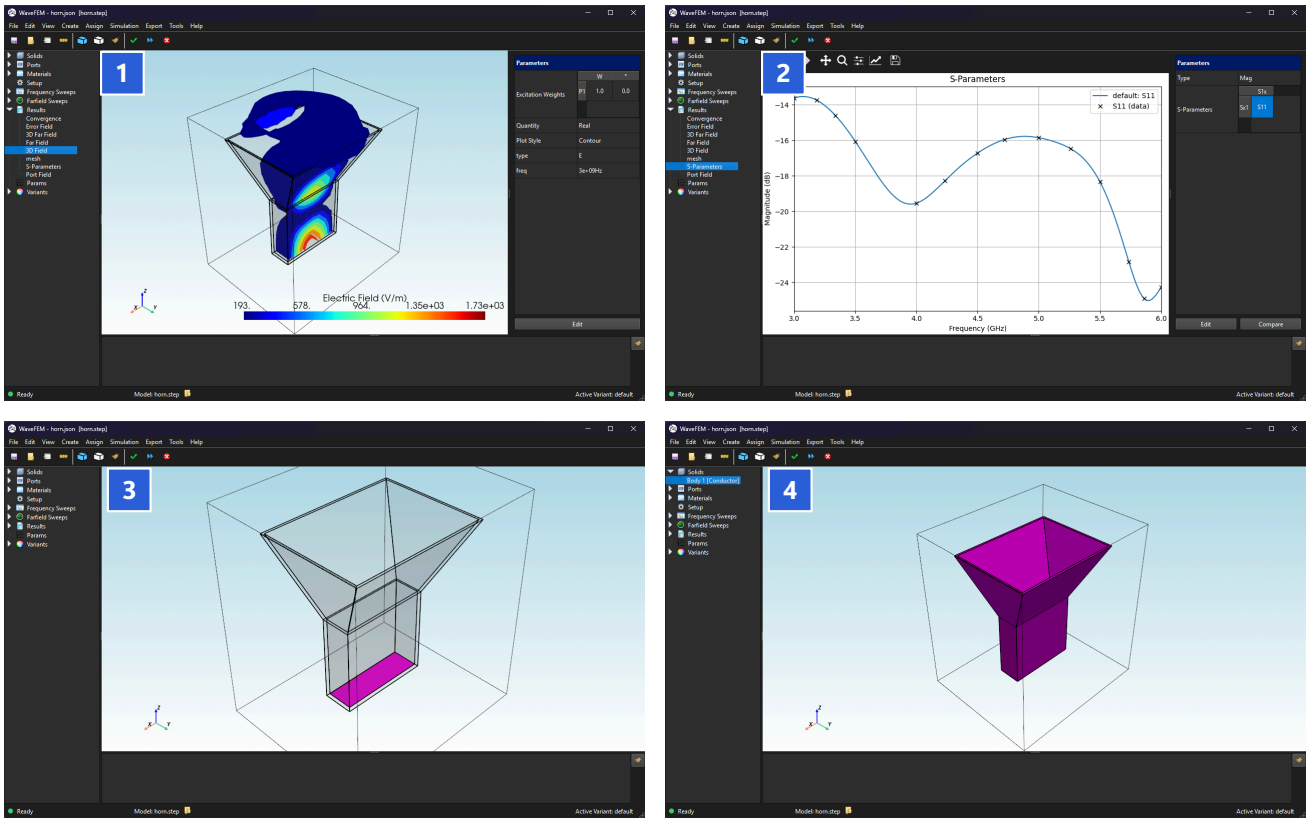


Figure 6 — Viewer modes: (1) 3D field, (2) 2D plot, (3) selected face, (4) selected solid

Console

The console shows the progress of a running simulation (see Run and stop). It also keeps a record of errors: whenever something fails, a dialog explains it and the same message is added to the console in red, so it can be read again after the dialog is closed.

Routine confirmations, such as a project being saved or a file exported, appear briefly in the status bar instead.



Figure 7 — Console after a simulation

Parameter tables

Settings are edited in tables like the one below. Related settings are grouped into tabs. Hover over a name for a description; greyed-out rows are disabled by another setting and their tooltip says which.

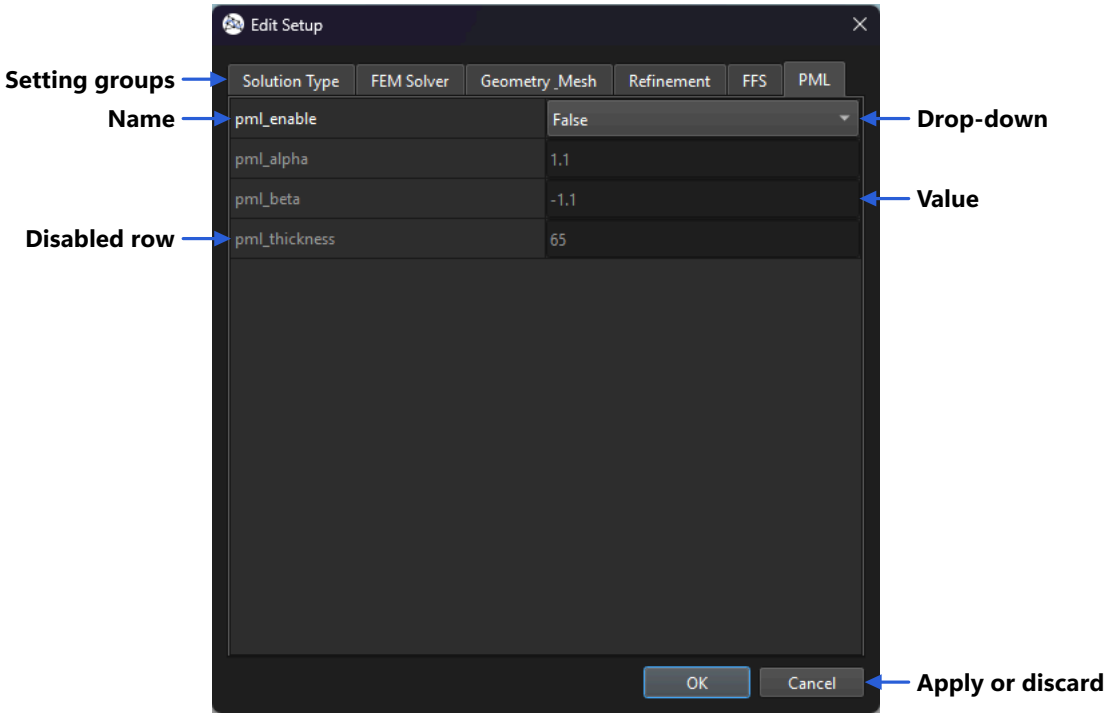


Figure 8 — A parameter table (**Edit Setup**, PML tab)

BACKGROUND

How WaveFEM Works

Physics

WaveFEM solves the source-free vector Helmholtz equation for the electric field:

$$\nabla \times \left(\frac{1}{\mu_r} \nabla \times \vec{E} \right) - k_0^2 \varepsilon_r \vec{E} = 0$$

where μ_r and ε_r vary through the model. The magnetic field is computed afterwards:

$$\vec{H} = \frac{\nabla \times \vec{E}}{-j\omega\mu}$$

The finite element formulation follows [1].

Solution procedure

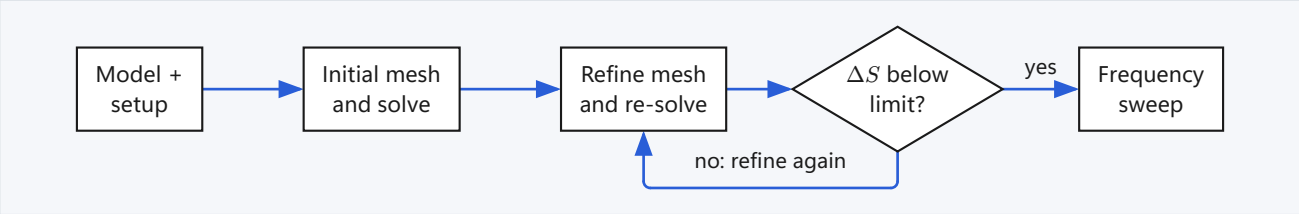


Figure 9 — Solution procedure

WaveFEM refines the mesh at one frequency until the S-parameters stop changing (ΔS below the limit). The final mesh is then used for every frequency in the sweeps. A sweep can solve fixed points or be **adaptive**: WaveFEM fits a rational function [2] to the solved points and adds points until the fit stops changing, then evaluates the fit across the band.

Ports

Ports are where energy enters and leaves the model.

- **Lumped ports** apply a voltage across a small rectangle between two conductors, such as the end of a microstrip trace and its ground plane.
- **Wave ports** inject the modes of a waveguide or transmission line cross-section. The face must be backed by a conductor.

Results

- **Electric and magnetic fields** in the model: where energy is stored, coupled or confined.
- **S-parameters**: reflection and transmission between ports, e.g. return loss (S_{11}) and insertion loss (S_{21}).
- **Far-field patterns**: directivity or gain against angle for radiating structures.
- **Convergence and error data**: how the adaptive mesh refinement progressed.

USING WAVEFEM

Workflow

A simulation follows the same steps every time. Each has its own section.

1. **Model** — import a STEP, DXF/GDS or Python file. Planar files need a layer stack-up.
2. **Materials** — create materials and assign one to every solid.
3. **Ports** — add wave ports on faces, sketch lumped ports, or draw layout ports on a planar design.
4. **Settings** — set the refinement frequency and mesh options, add frequency sweeps and, for antennas, far-field sweeps.
5. **Validate and run** — check the setup, save, run, and watch the console.
6. **Results** — plot fields, S-parameters and patterns; export what you need.
7. **Iterate** — create variants, change the design and compare results.

USING WAVEFEM

Importing Models

Choose **File** › **Import model file** and select the file. Pick the file type at the bottom right of the dialog.

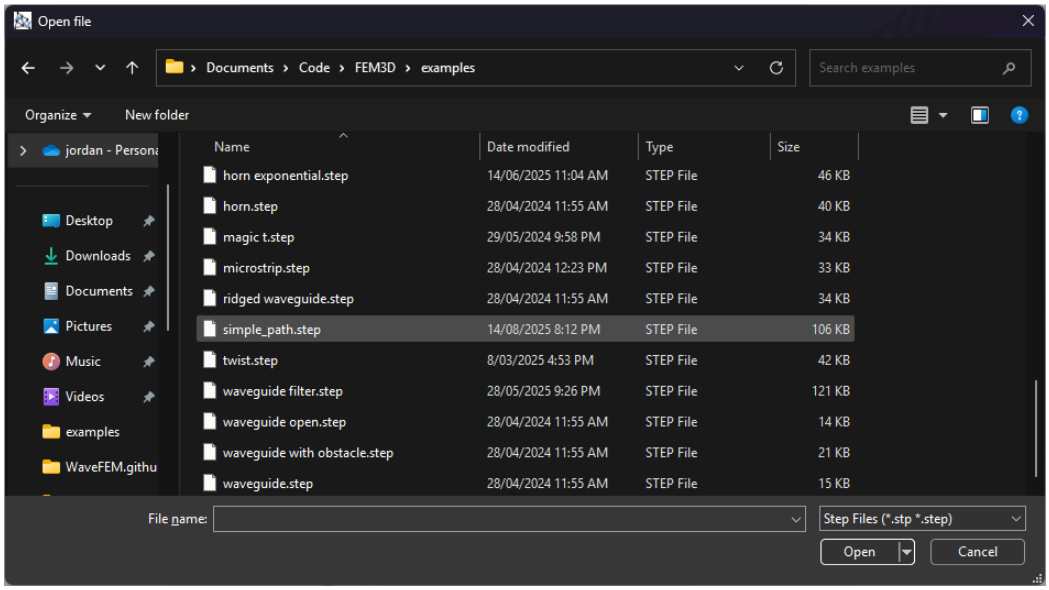


Figure 10 — Selecting a model file

Note Saving the project copies the model file next to the project file, so the project stays self-contained. The original file is not changed.

STEP files

A STEP file loads straight into the viewer. Each body becomes a solid in the tree.

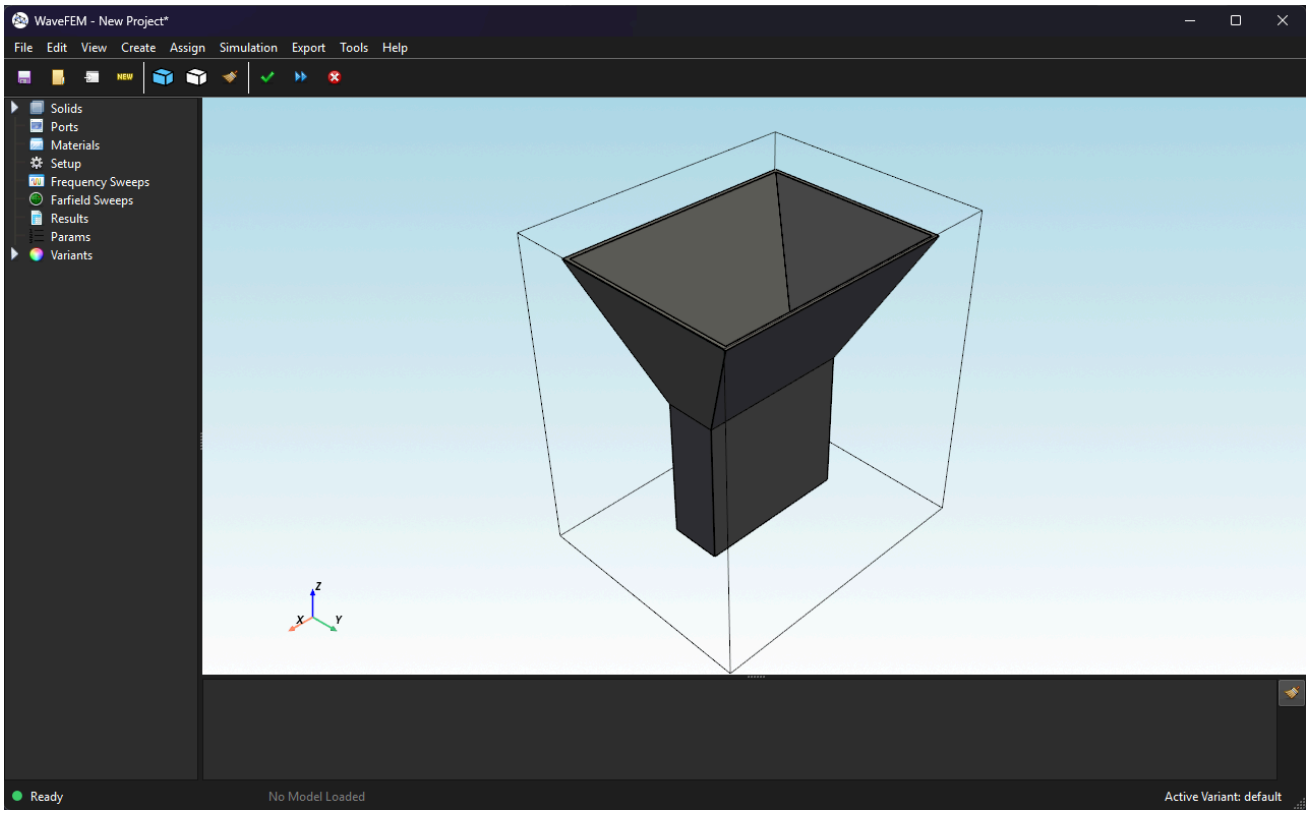


Figure 11 — Imported STEP model

Planar files (DXF and GDS)

DXF (PCB) and GDS (IC) files are 2D, so WaveFEM asks how to build them into 3D with the **Layer Stack Configuration** window.

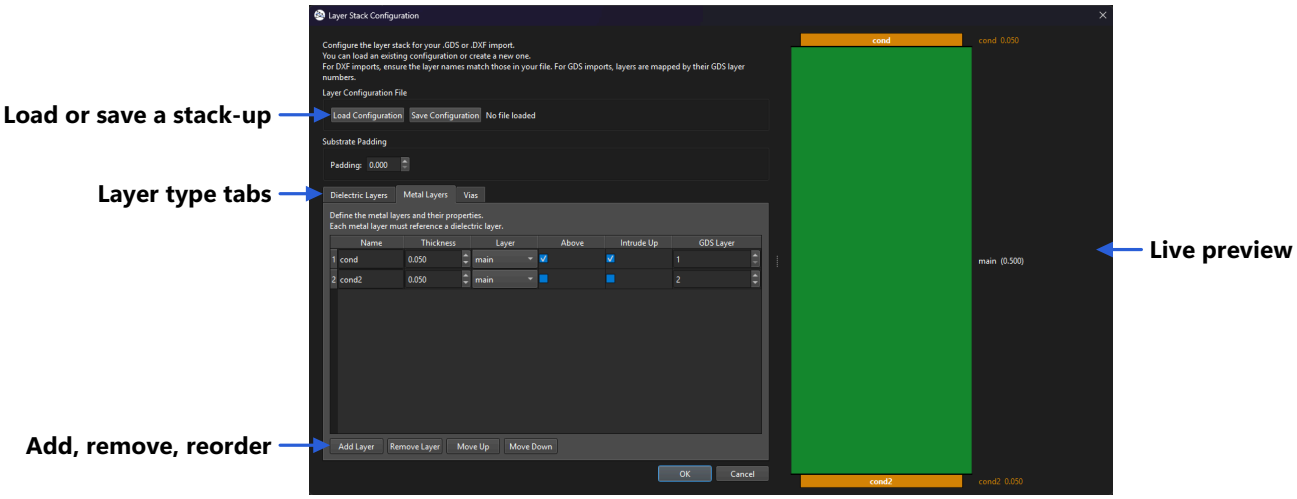


Figure 12 — Layer Stack Configuration

Dielectric layers	Substrates, listed bottom to top, each with a name and thickness.
Metal layers	Conductors. Each has a name, a thickness, the dielectric it sits on, and the layer in the file it takes its shapes from. **Above** and **Intrude Up** place it; see below.
Vias	Vertical connections through a dielectric, taken from a file layer.
Padding	Extra substrate around the drawn shapes.

Placing a metal layer. A metal layer starts as the shapes from the file, drawn on one surface of its dielectric, and is then extruded by its thickness:

- **Above** picks the surface: ticked for the top of the dielectric, unticked for the bottom.
- **Intrude Up** picks the direction of the extrusion from that surface: ticked extrudes up, unticked extrudes down.

So on the top surface, **Intrude Up** ticked gives metal standing on the dielectric and unticked sinks the metal into the dielectric, flush with its top. On the bottom surface, ticked pushes the metal up into the dielectric and unticked hangs it below.

Matching file layers. For a GDS file, each metal layer lists the GDS layer number(s) it takes its shapes from. For a DXF file, the layer is matched by name: the metal layer's name must be exactly the name of the DXF layer, and any GDS layer number is ignored.

The stack-up is saved in the project. **Save Configuration** writes it to a `.json` file for use in other projects.

> **Note** The DXF importer reads polylines (`LWPOLYLINE`, `POLYLINE`, including arc segments), circles, and outlines drawn from separate `LINE`, `ARC`, `ELLIPSE`, `SPLINE` and open polyline pieces whose ends meet. Arcs, circles and splines are imported as true curves, not polygons. An open polyline that meets nothing is closed with a straight edge.

> **Note** GDS holds only polygons, so a curved edge becomes many short straight segments, each a separate face. If the design has curves (round patches, arcs, spirals, tapers), DXF may be the better format: the curves stay smooth, the model has far fewer faces, and the mesh along the edges is cleaner.

Python scripts

A Python file that builds a CadQuery model loads like a STEP file. A script can instead write a planar layout with gdstk (`out_file` ending in `.gds`) or ezdxf (`out_file` ending in `.dxf`); WaveFEM then asks for the stack-up, as for a GDS or DXF file. Prefer ezdxf when the layout has curves, for the reasons above; its metal layers are matched to DXF layers by name. Variables defined at the top of the script appear under **Params** changing one rebuilds the model immediately. Parameters are stored per variant.

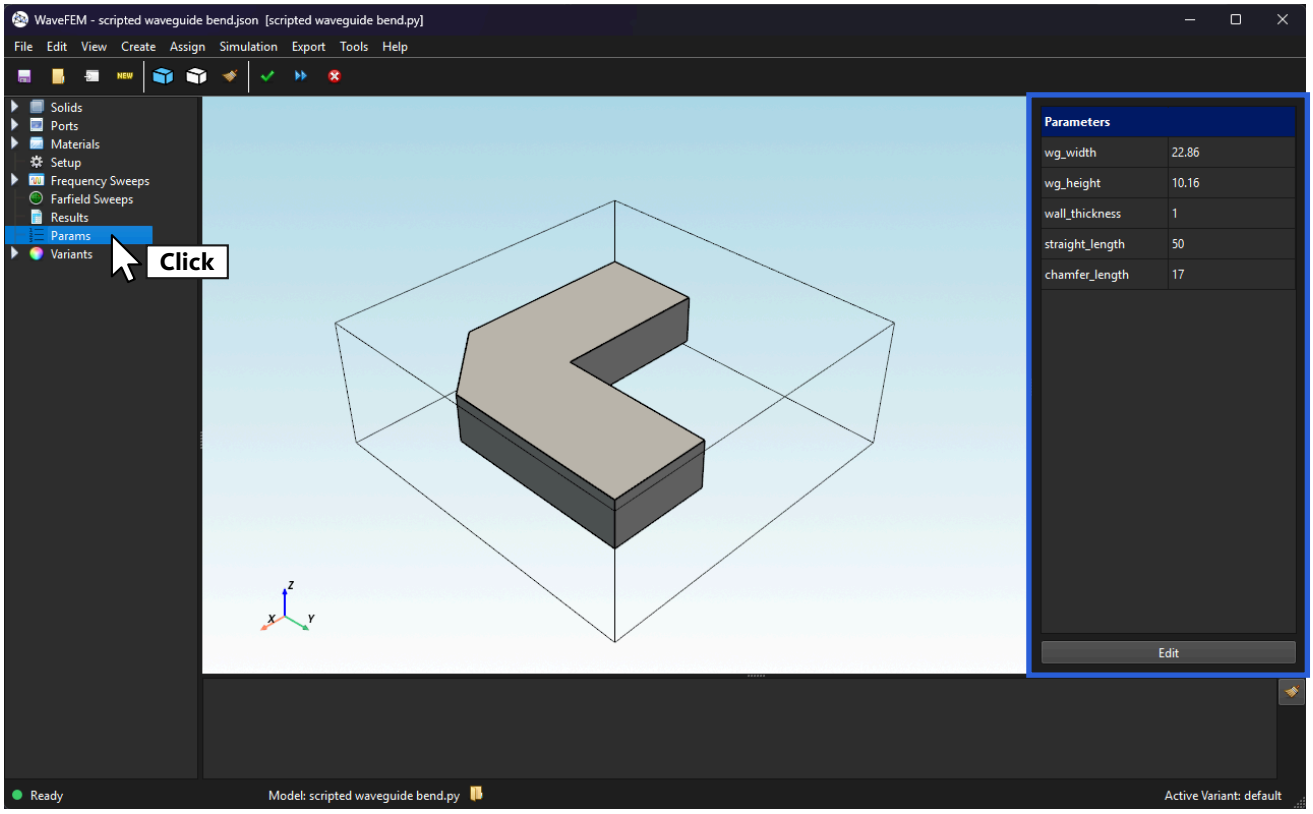


Figure 13 — Parameters of a scripted model

If the script fails, a dialog shows the Python error, and the console keeps a copy, for example:

```
Error loading model:

unexpected indent (<cqscript>, line 12)
```

> **Note** Debug scripts in a Python editor first; errors are easier to trace there.

Saving and opening projects

Projects are `.json` files: **File** › **Save** and **File** › **Open**. Keep the model file in the same folder as the project; if it is missing when a project opens, WaveFEM asks you to locate it.

USING WAVEFEM

Materials

Every solid needs a material before a simulation can run.

Creating a material

Choose **Create** › **Material**, then copy one from the library or start from scratch.

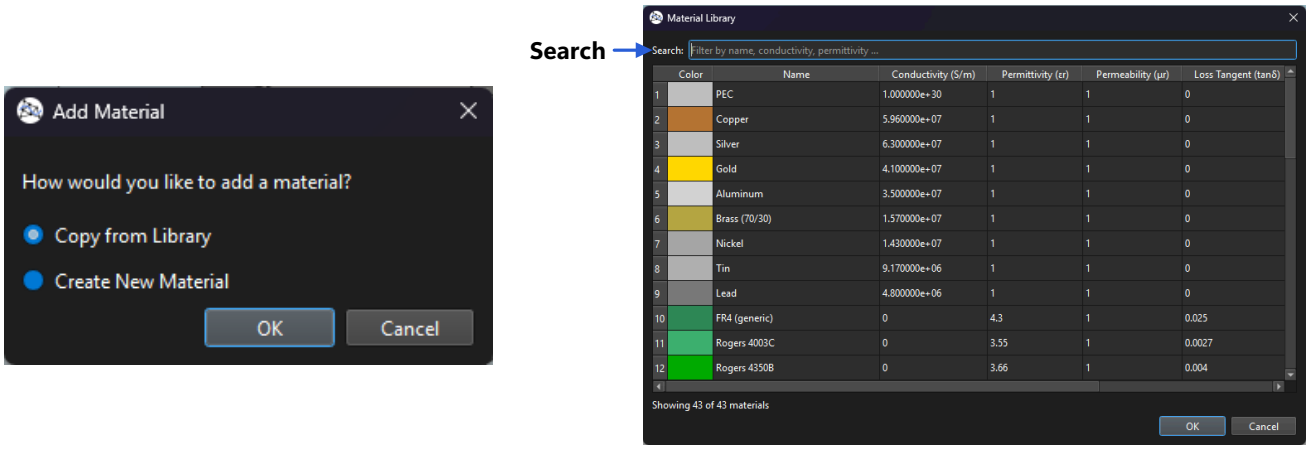


Figure 14 — Adding a material: choice dialog and library

The material opens in a parameter table:

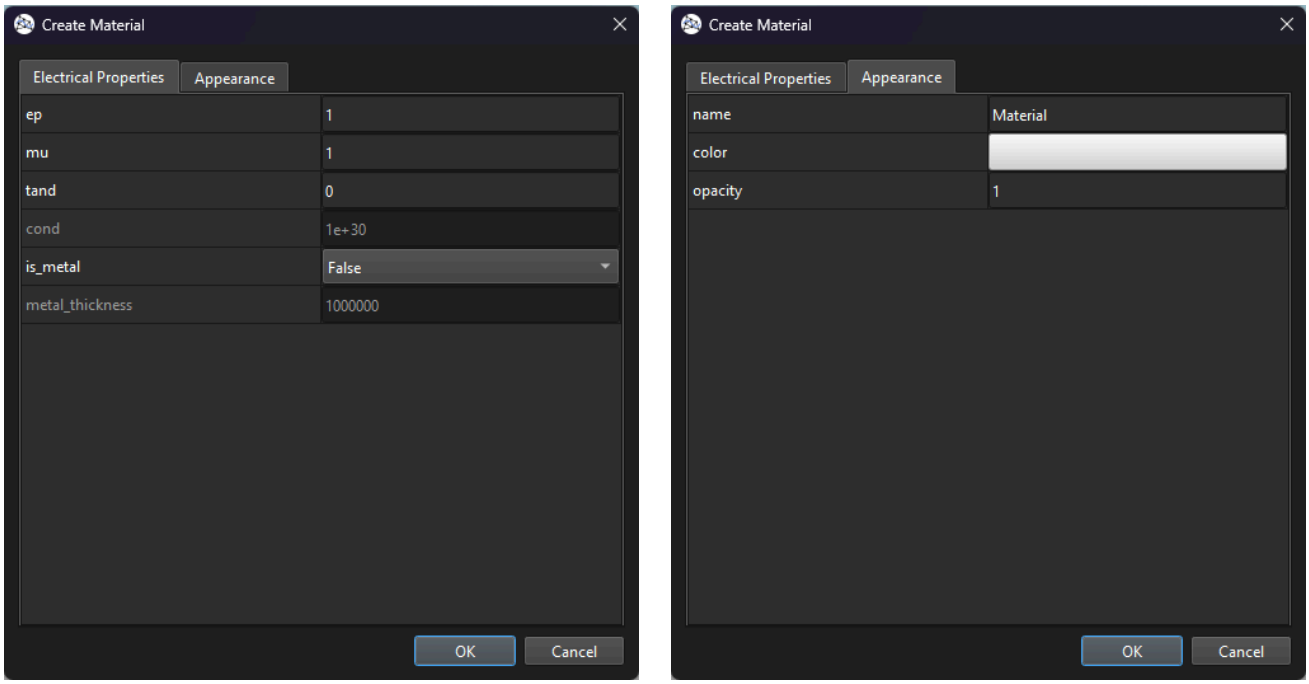


Figure 15 — Material properties

`ep`	Relative permittivity.
`mu`	Relative permeability.
`tand`	Dielectric loss tangent.
`is_metal`	Treat as a metal (see the note below).
`cond`	Conductivity in S/m (metals).
`metal_thickness`	Physical thickness of a metal, used when it is comparable to the skin depth.

`name`	Shown in the tree next to each solid that uses it.
`color, opacity`	Display colour and transparency (1 solid, 0 clear).

> **Note** Metals are modelled with a surface impedance boundary and are not meshed inside. This is accurate when the skin depth is much smaller than the metal thickness. For thin or poorly conducting metals set `metal_thickness`.

Assigning materials

Click a solid in the tree (hold **Ctrl** or **Shift** to select several), then choose **Assign** › **Material** and pick the material. The tree then shows the material next to the solid, e.g. `Body 1 [Conductor]`.

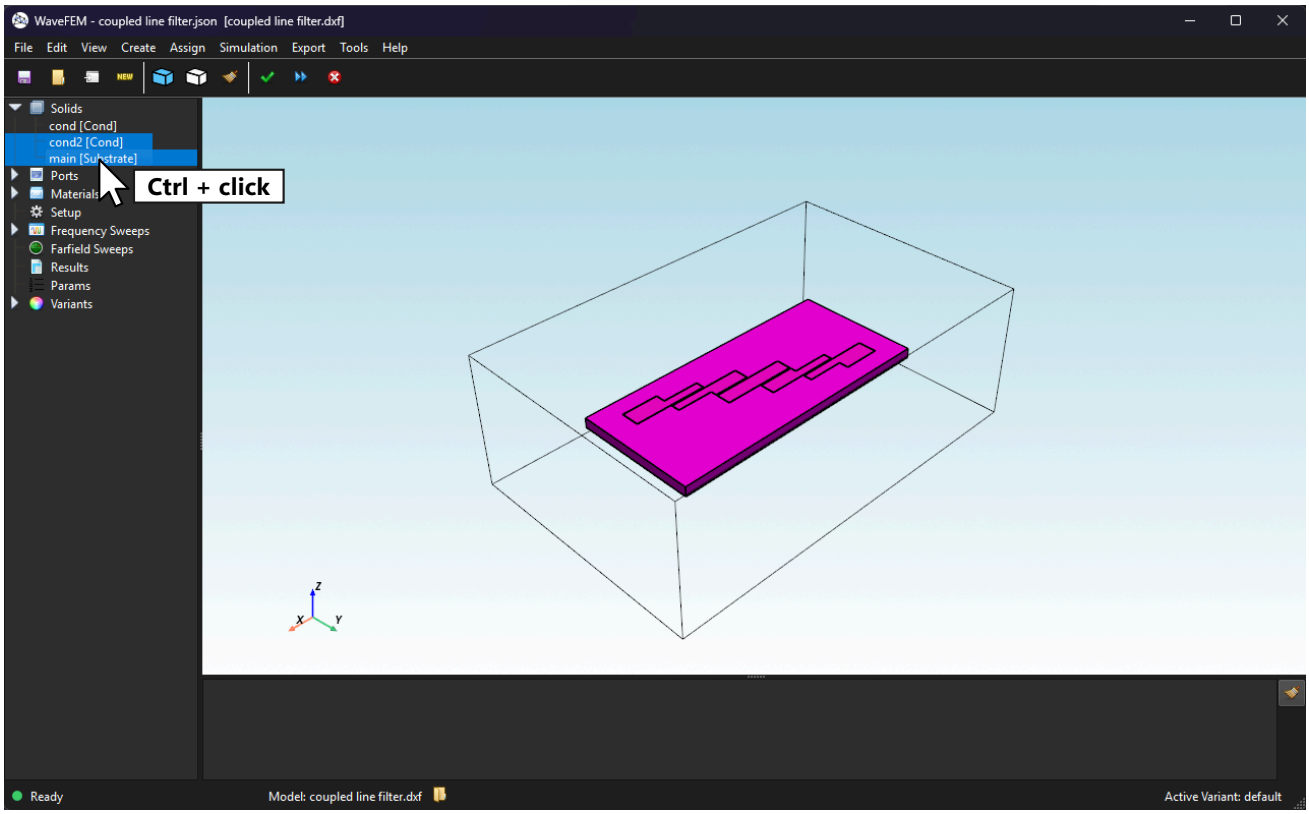


Figure 16 — Two solids selected

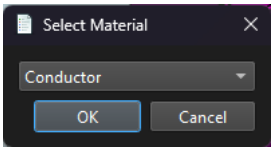


Figure 17 — Assign › Material

USING WAVEFEM

Ports

Wave ports

1. Right-click the port face in the viewer. If another face is in front, press `B` (**Edit › Select Behind**) until the right face is highlighted.
2. Choose **Assign › Wave Port**.
3. Check it: expand **Ports** and click the port to highlight it.

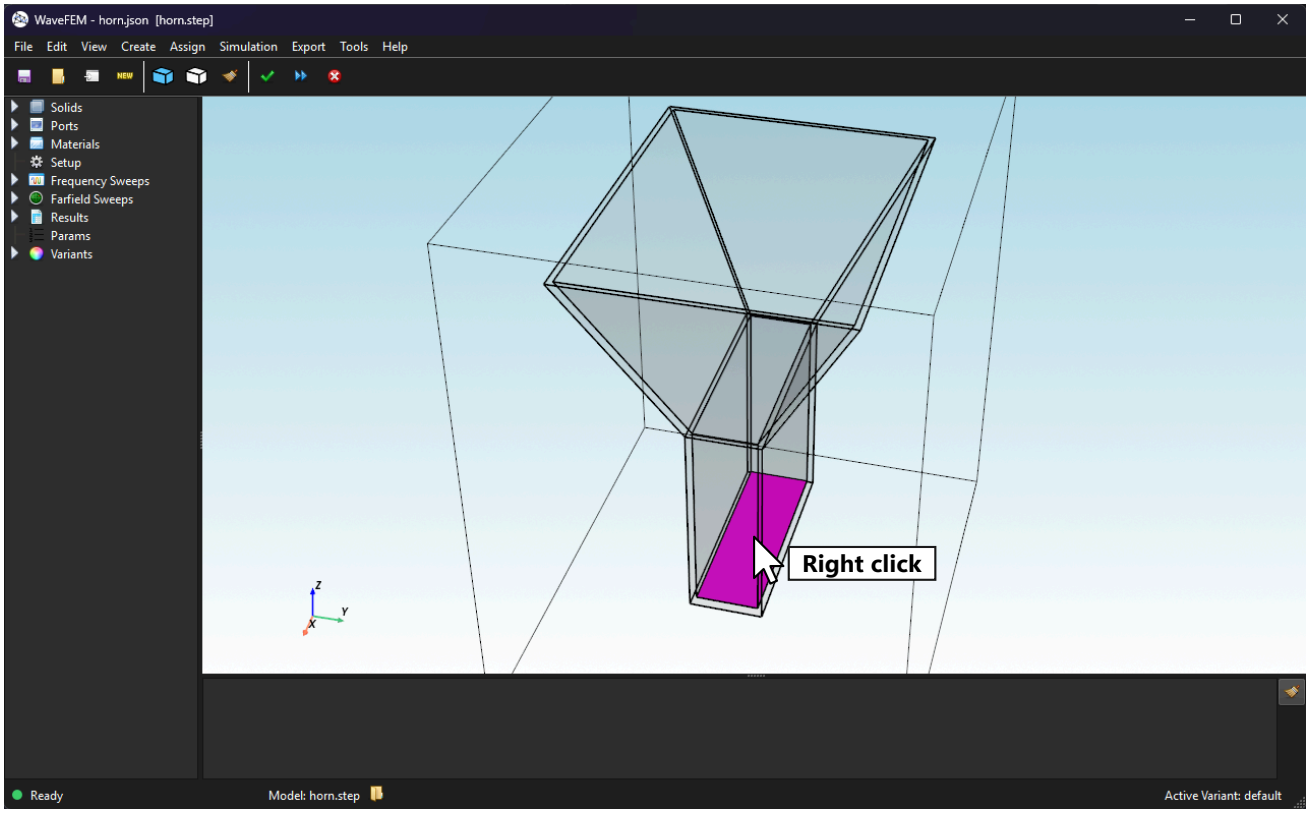


Figure 18 — Selecting a wave port face

Lumped ports

A lumped port is a rectangle drawn on a plane between two conductors.

1. Right-click a face that lies in the port's plane (here, the end of the feed line).
2. Choose **Create › Sketch Lumped Port** and keep **Parallel to face**.
3. In the **Sketch Drawer**, click **Rectangle**, then click the two corners. Points snap to the grid, corners and intersections. You can draw helper rectangles, but delete them so that exactly one remains.
4. Click **OK** and enter the port impedance.

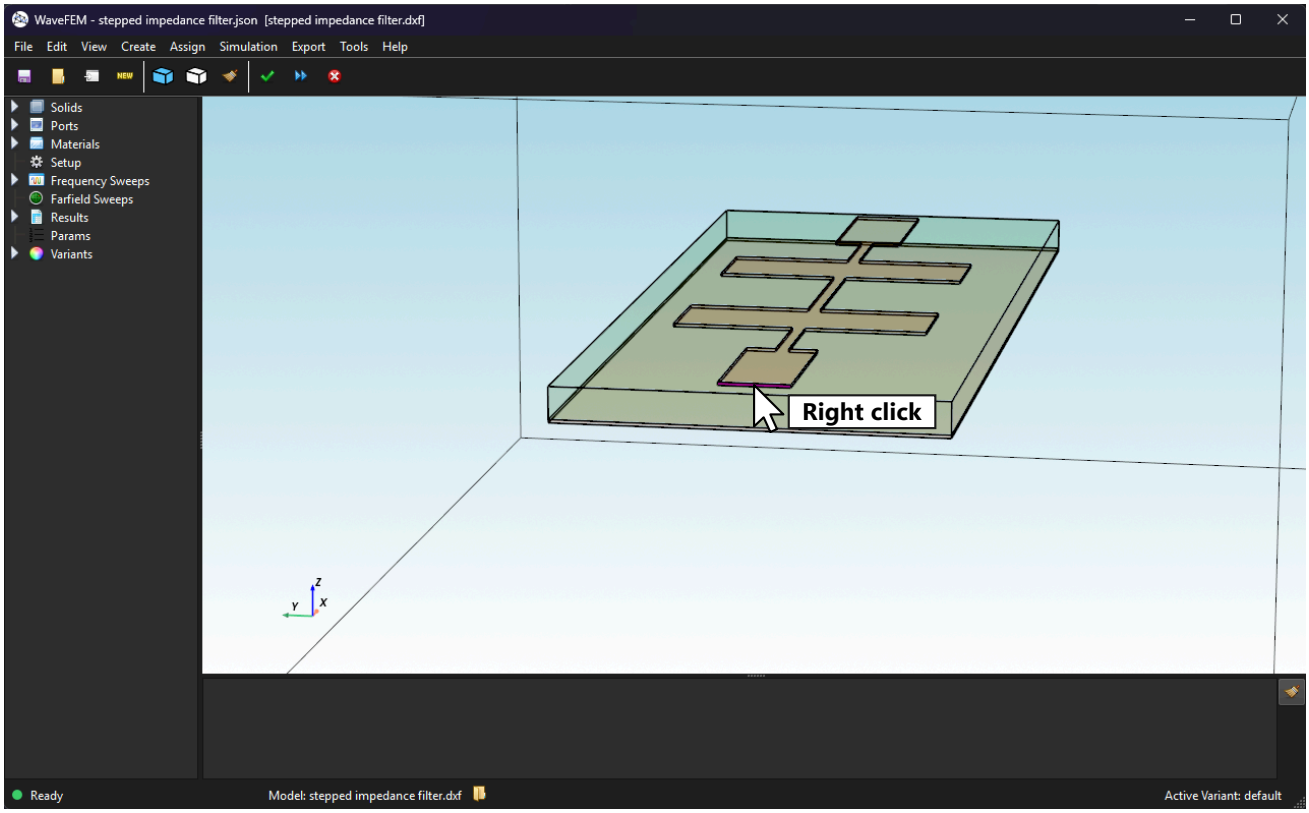


Figure 19 — Step 1: select a face in the port plane

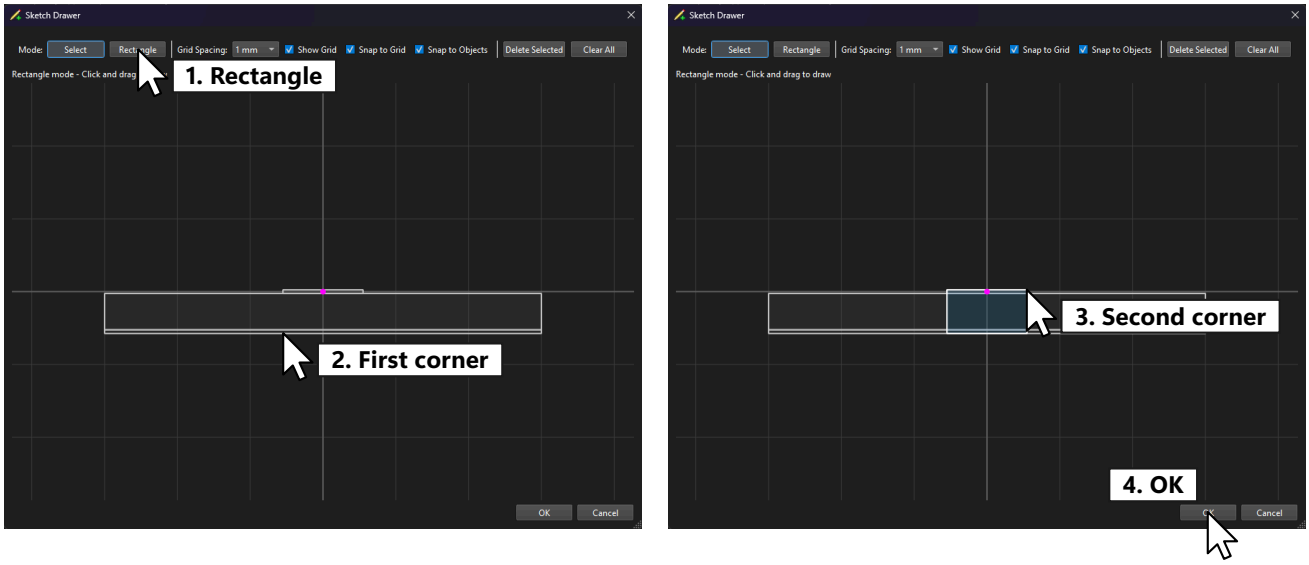


Figure 20 — Steps 3–4: drawing the port

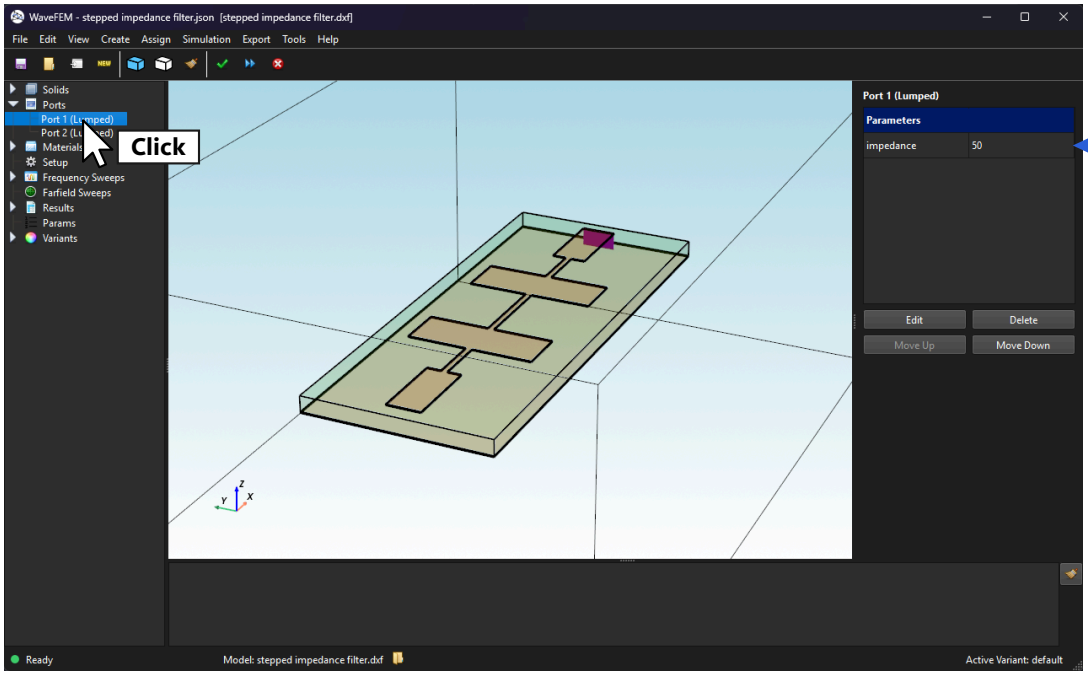


Figure 21 — The new port, selected in the tree

> **Note** Lumped ports must be rectangles.

Layout ports

On a planar (DXF or GDS) design a lumped port can be drawn on the layout instead of sketched on a face. A layout port is a line in the layout plus a top and a bottom metal layer. WaveFEM builds the port rectangle from the stack-up: along the line, from the bottom face of the bottom layer to the top face of the top layer. Because it is not tied to a face, the port moves with the stack-up when layer thicknesses or the **Above** and **Intrude Up** options change, and it is kept when the model is rebuilt.

1. Choose **Edit › Layout Ports...** (`Shift+P`). The editor shows the metal layers from above.
2. With **Pick Edge** selected, click the edge of the trace where the port goes. The whole edge becomes the port. The edge's layer is used as the top layer and a layer below it as the bottom layer.
3. Repeat for every port, then select each in the **Ports** list and check its layers and impedance.
4. Click **OK**. Every change made in the editor is applied together.

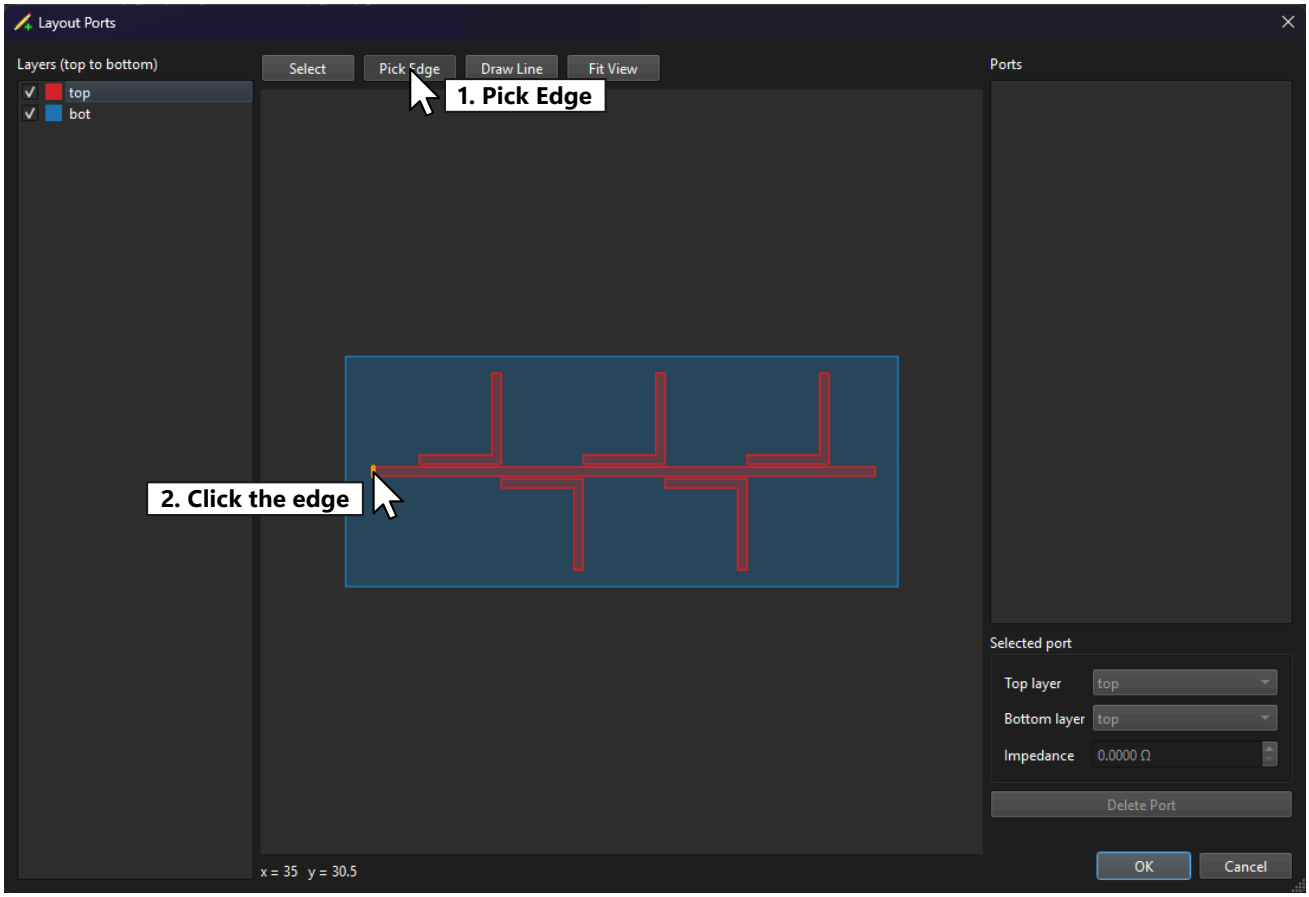


Figure 22 — Picking the edge at the end of the through line

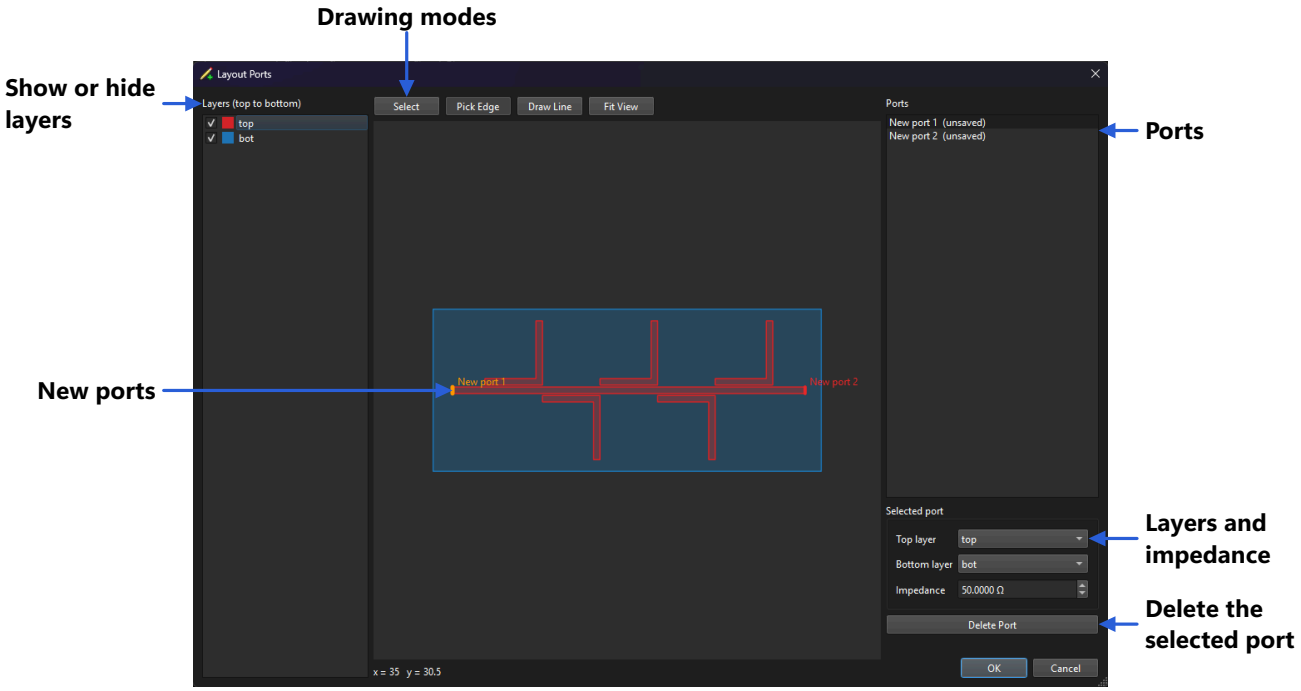


Figure 23 — Two layout ports, one at each end of the line

Select	Click a port on the layout to select it.
Pick Edge	Click a polygon edge to place a port along the whole edge.
Draw Line	Click the two ends of the port. Points snap to vertices, edge midpoints and edges. `Esc` cancels a line.
Fit View	Show the whole layout. The mouse wheel zooms; drag with the middle or right button to pan.
Layers	Untick a layer to hide it, so that clicks only pick edges of the layers you can see.
Top layer, Bottom layer	The metal layers the port connects. The bottom layer must lie below the top layer.

In the tree a layout port is named after its layers, e.g. `Port 1 (Layout bot→top)`. **Edit in Layout…** in its properties panel reopens the editor with that port selected.

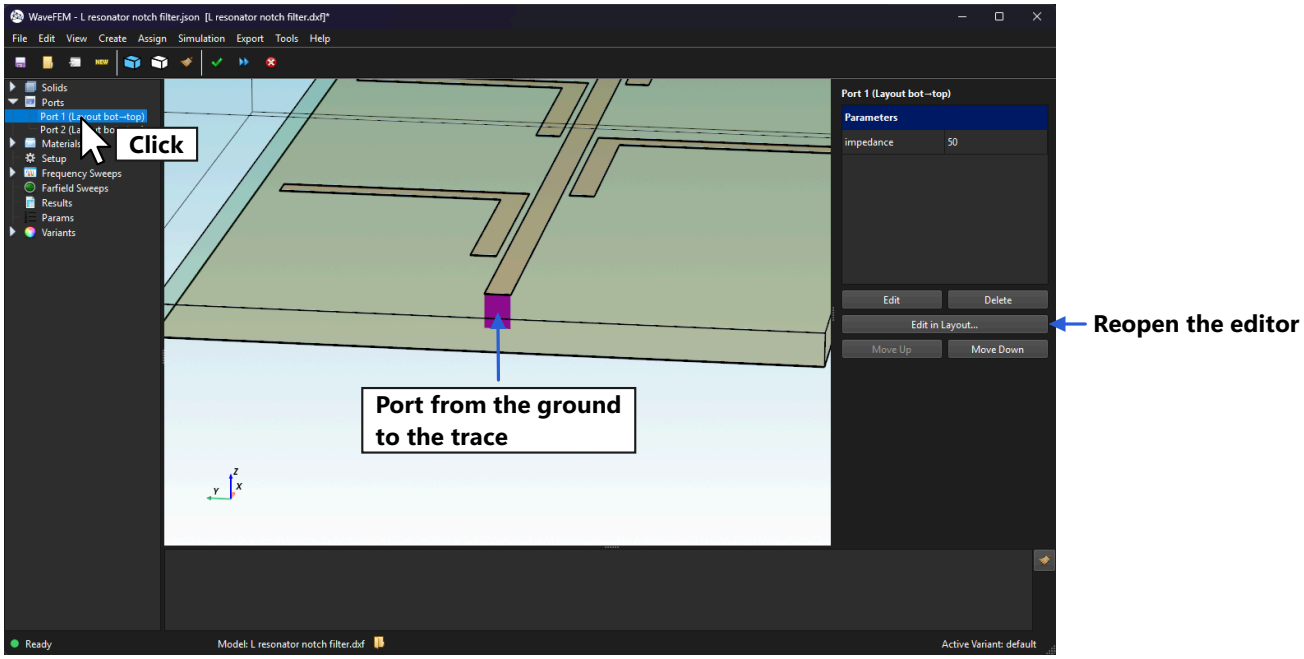


Figure 24 — A layout port between the ground plane and the trace

Note If the stack-up no longer has a port's layer, the port is marked `- invalid` and the setup does not validate until the port is fixed in the editor.

USING WAVEFEM

Simulation Settings

Solver settings

Click **Setup** in the tree, then **Edit**. The most important setting is `refine_frequency`: choose a frequency in or near the top of your band.

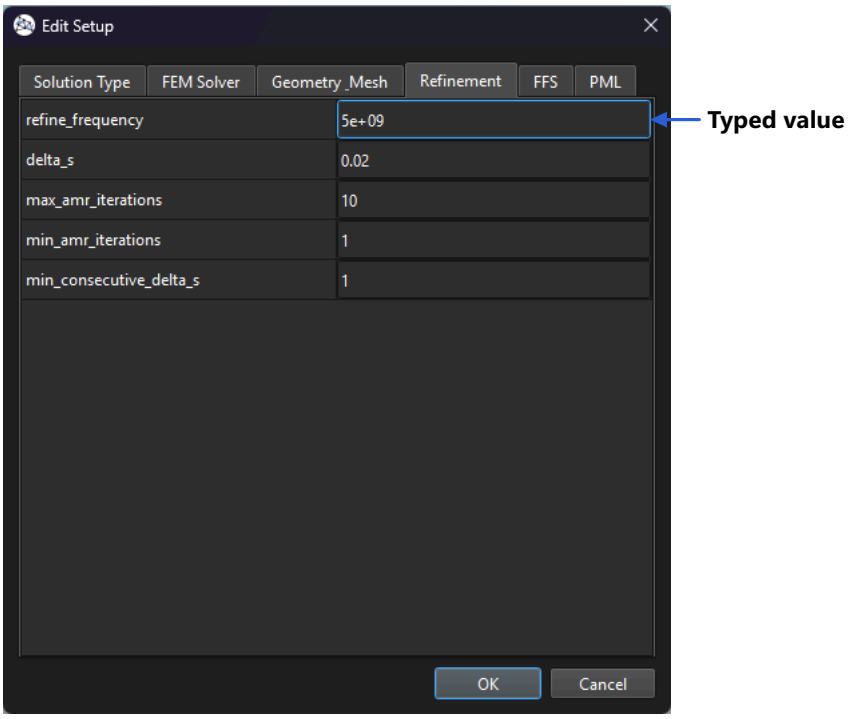


Figure 25 — Refinement settings

`base_unit`	Model length unit: nm, µm, mm, cm, m, mil or in.
`bounding_box_padding`	Air added around the model, in model units. Aim for about a quarter wavelength.
`target_elements_per_wavelength`	Initial mesh density.
`port_element_density`	Initial mesh density on ports.
`refine_frequency`	Frequency (Hz) at which the mesh is refined.
`delta_s`	Convergence limit for adaptive refinement.
`max_amr_iterations / min_amr_iterations`	Bounds on the number of refinement passes.
`min_consecutive_delta_s`	Passes in a row that must meet `delta_s`.
`max_ffs_iterations`	Maximum points in an adaptive frequency sweep.
`ffs_fit_tol`	Convergence tolerance of the adaptive sweep fit.
`pml_enable`	Absorb outgoing waves with a perfectly matched layer outside the box.
`pml_alpha, pml_beta`	Decay rate and propagation speed in the PML.
`pml_thickness`	PML thickness, in model units.
`cudss`	Solve on an NVIDIA GPU with cuDSS instead of the CPU. Needs the extra software in GPU solver.

`cudss_hybrid_execute`	Let cuDSS run parts of the factorization and solve on the CPU as well as the GPU.

Frequency sweeps

Create › **Sweep** adds a sweep. A project can have several, e.g. a wide adaptive sweep for S-parameters plus a single frequency that saves fields.

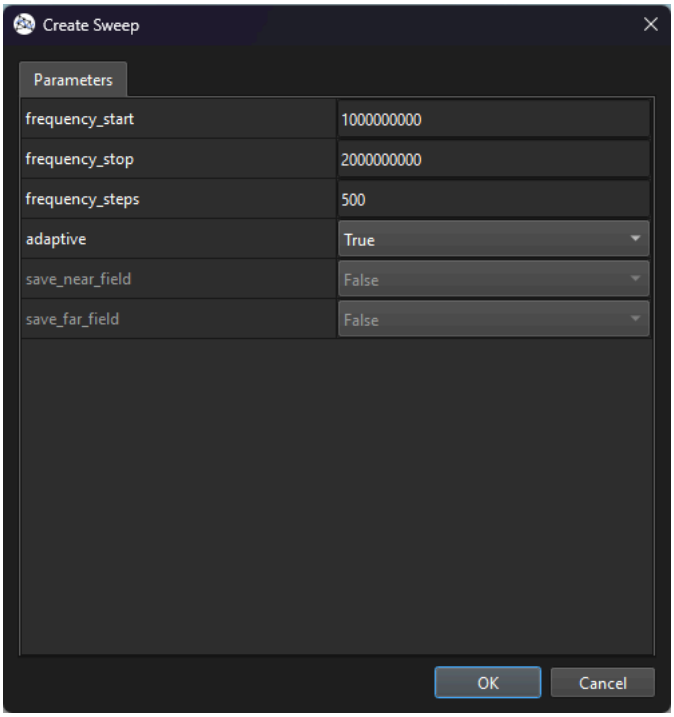


Figure 26 — Frequency sweep

`frequency_start, frequency_stop`	Band edges in Hz. Do not start at 0 Hz; use 1 Hz instead.
`frequency_steps`	Number of output points (for an adaptive sweep, the points the fit is evaluated at).
`adaptive`	Choose solve points automatically until the response is resolved.
`save_near_field`	Save fields inside the model.
`save_far_field`	Save radiation patterns (needs at least one far-field sweep).

Far-field sweeps

Create › **Farfield Sweep** defines a radiation pattern cut or a full 3D pattern.

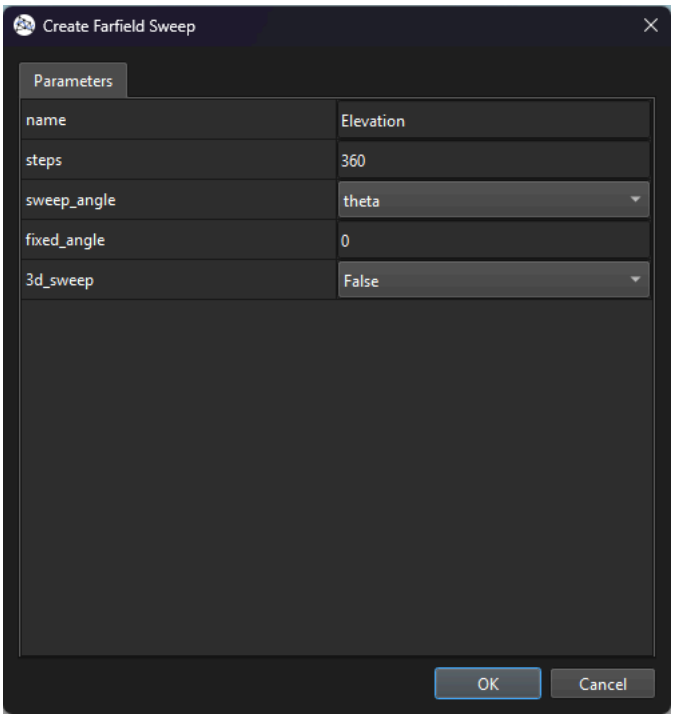


Figure 27 — Far-field sweep

`name`	Name shown in the tree and plots.
`steps`	Points per revolution. Keep this low (about 50) for 3D sweeps, which use many revolutions.
`sweep_angle`	The angle the cut sweeps: theta (at fixed phi) or phi (at fixed theta).
`fixed_angle`	The angle that is held fixed (degrees; not used in 3D).
`3d_sweep`	Compute the full 3D pattern.

USING WAVEFEM

Running Simulations

Validate

Simulation › **Validate** checks the setup. A setup needs at least one port, a material on every solid, and at least one frequency sweep. A valid setup is confirmed with **Setup is valid!**; otherwise a dialog lists every problem, and the list is also added to the console:

```
Setup is invalid:

- No ports defined!
- Material assignments not set.
```

Simulation › **Run** performs the same check first and does not start until the problems are fixed.

Run and stop

Simulation › **Run** saves the project and starts the solver; only one simulation runs at a time. **Simulation** › **Stop** cancels it. Progress appears in the console:

```
===== Started Running at 19:31 =====
Adaptive Mesh Refinement: 6e+09Hz
 Iteration Num Elems  Time (s)   Delta S  Peak Memory (mb)
         1       868         0                        64
         2       931         0     0.254               85
         3      1152         0     0.111               94
         4      1384         0    0.0236              111
         5      1772         0  0.006995              134
Convergence reached (Delta S < 0.02).
Frequency Sweep:
 Frequency      Time
     3e+09         0
     6e+09         0
   4.5e+09         0
===== Finished Successfully at 19:31 =====
```

The first table is the mesh refinement: elements grow until ΔS falls below the limit. The second lists the frequencies solved by the sweep.

If the solver fails, a dialog shows its error message and the console keeps a copy in red. A run stopped with **Simulation** › **Stop** ends with **Simulation terminated by user.** in the console.

USING WAVEFEM

Results

When a run finishes, its results appear under **Results** in the tree. Click one to plot it. The properties panel shows the plot settings; **Edit** changes them.

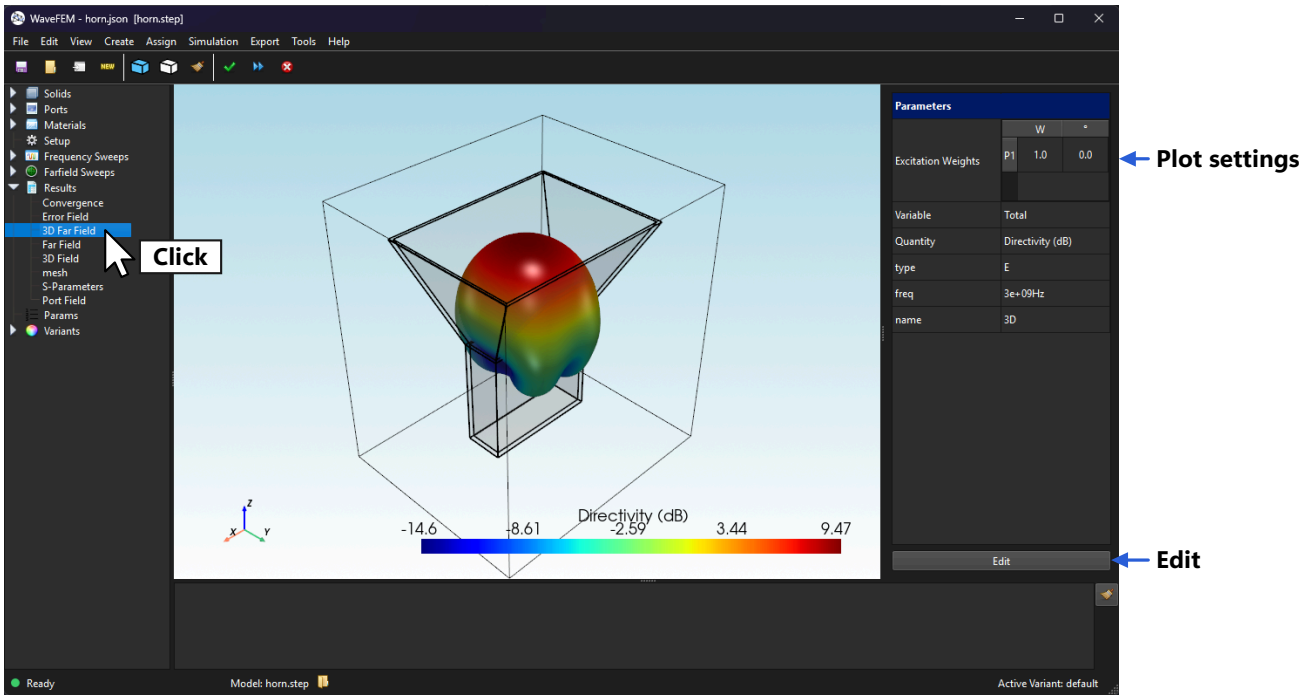


Figure 28 — Plotting a result

Changing what is plotted

1. Click **Edit** under the plot settings.
2. Change the settings, e.g. the frequency.
3. Click **OK**.

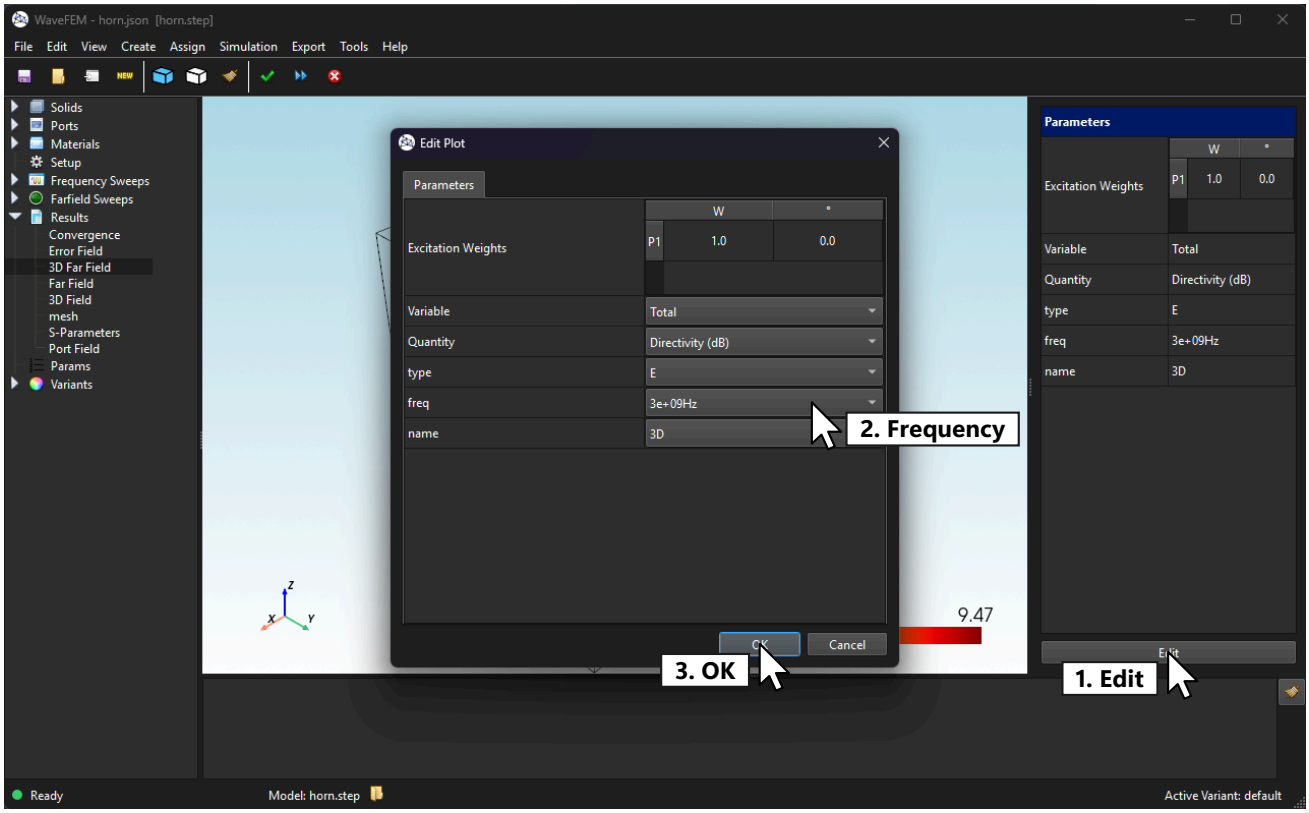


Figure 29 — Editing plot settings

Excitation weights

Field results can be shown for any combination of port excitations. The **Excitation Weights** table sets the power (W) and phase (degrees) of each port; edit it through **Edit** by double-clicking a value. Exciting only one port of two horn antennas, for example, shows the field in that horn alone; give both ports a non-zero power to see both at once.

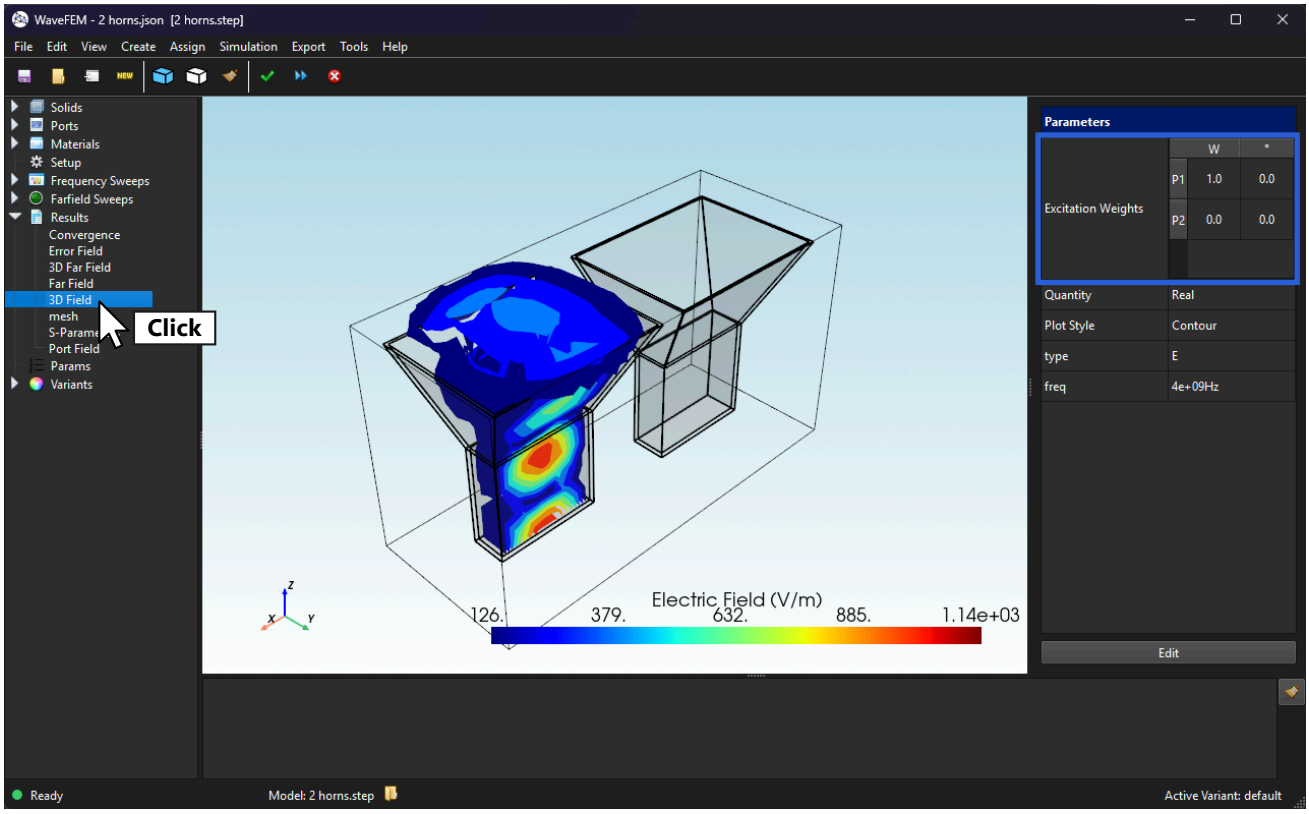


Figure 30 — Excitation weights for a two-port model

S-parameters

The S-parameter plot shows the parameters selected in its settings. In the edit dialog, click cells of the matrix to toggle them.

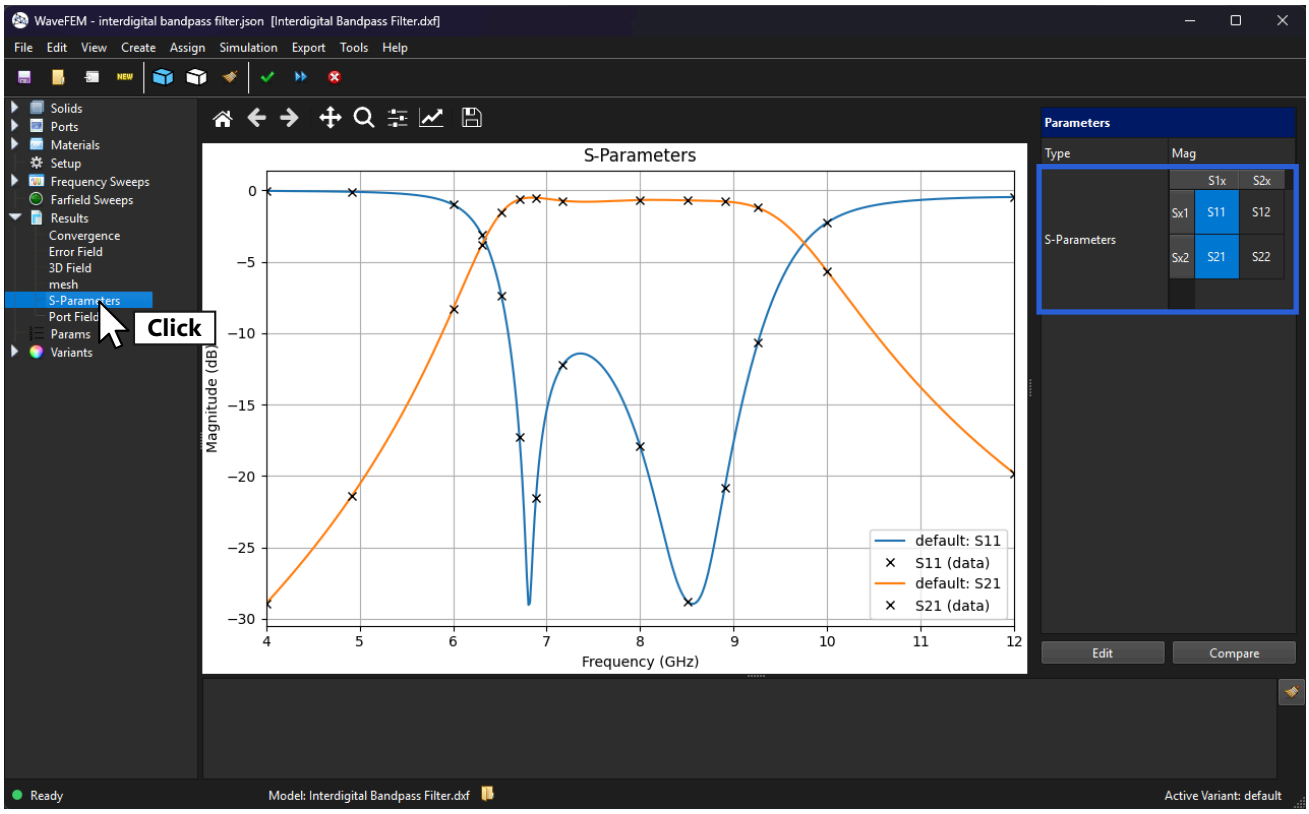


Figure 31 — S-parameters of a bandpass filter

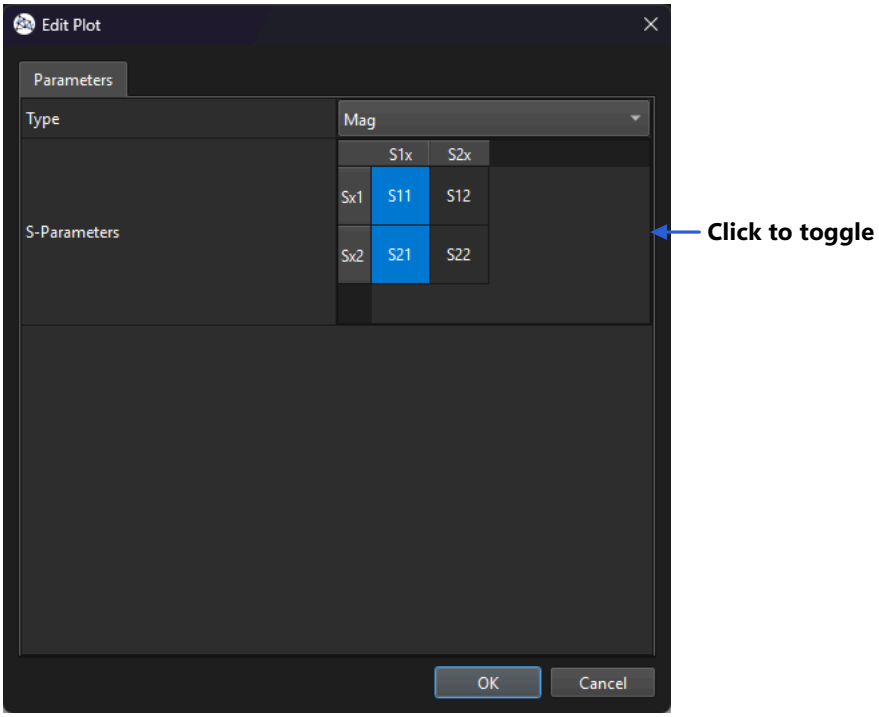


Figure 32 — Choosing S-parameters

Exporting

Display the result first, then use the **Export** menu.

Plot Data	The displayed data as `.csv` or `.xlsx`.
Plot Image	The displayed plot: `.png`, `.jpg` or `.tiff` for 3D plots; `.png`, `.svg` or `.pdf` for 2D plots.
Network Parameters	Touchstone `.sNp` file of the S-parameters.
.Step File	The 3D model, e.g. of a scripted or layered design.

USING WAVEFEM

Design Variants

A variant is one version of the design: its own model file or script parameters, ports, materials and results. Every project starts with a `default` variant and always has at least one. Expand **Variants** and select one to copy, rename, delete or activate it.

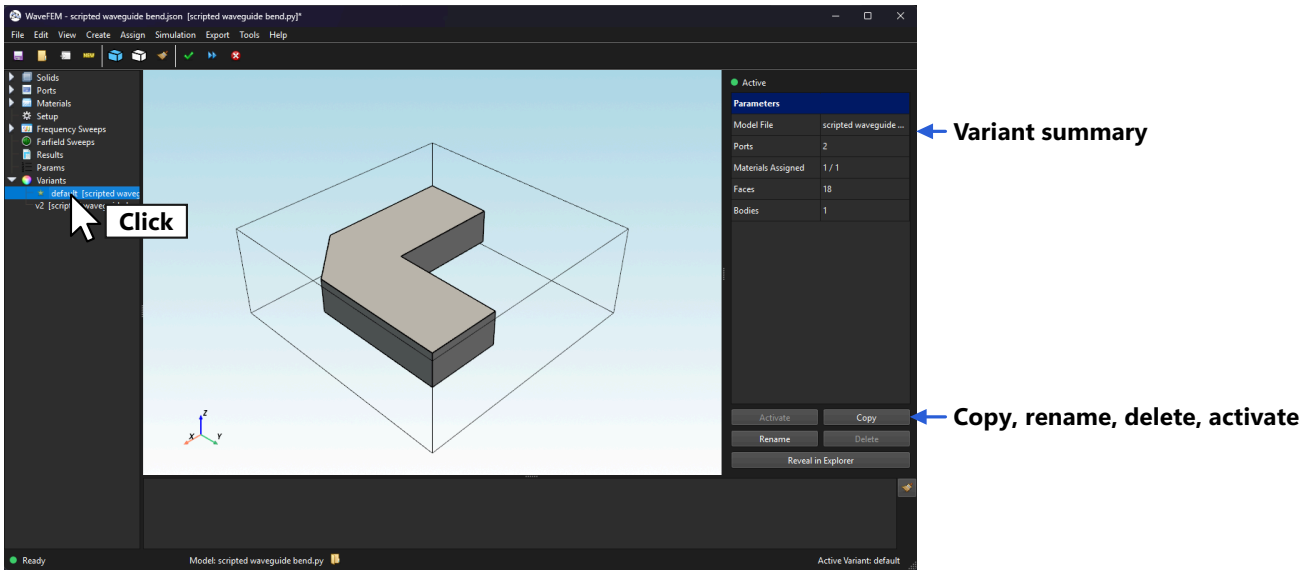


Figure 33 — Variants of a scripted waveguide bend

To compare variants, plot S-parameters or a far-field result, click **Compare** and pick the other variant. Both are drawn on the same axes.

USING WAVEFEM

AI Agents

An AI agent such as Claude Code or Cursor can work on the project open in the window through the Model Context Protocol (MCP). It can build or import a model, add materials, ports and sweeps, run the simulation and read the results. Each change appears in the window as the agent makes it, and you can keep working alongside it.

Starting the server

Open **Tools** › **MCP Server...** and click **Start**.

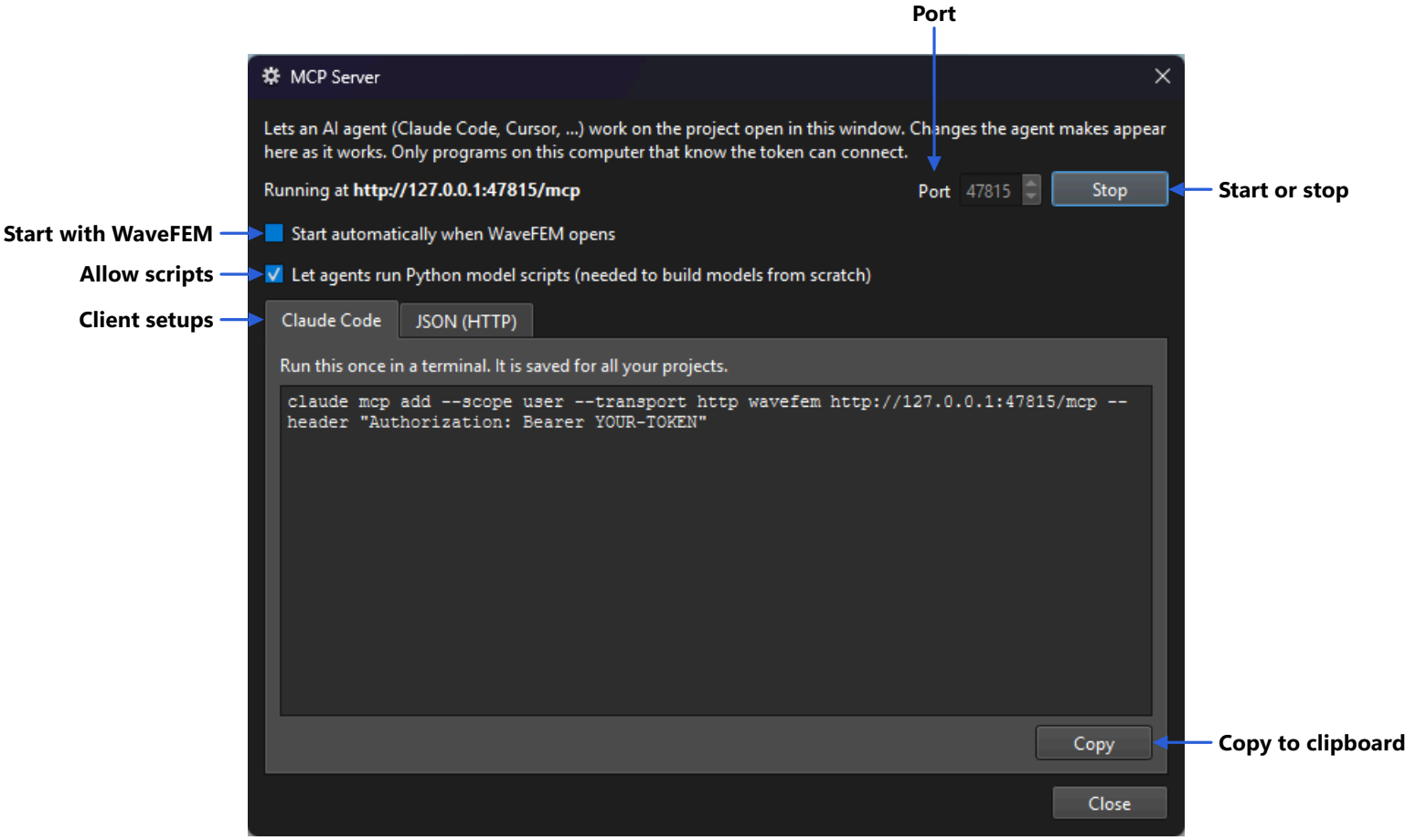


Figure 34 — The MCP Server window (the token is a placeholder)

Port	The server only accepts connections from this computer. Change the port while the server is stopped if another program already uses it (default 47815).
Start with WaveFEM	Start the server every time WaveFEM opens. Off by default.
Allow scripts	Lets the agent write and run Python model scripts, which it needs to build a model from scratch. Scripts run with your permissions; untick this to limit the agent to model files that already exist.
Client setups	Ready-made settings for connecting an agent, with a **Copy** button.

Connecting an agent

- **Claude Code:** copy the command on the **Claude Code** tab and run it once in a terminal. It is saved for all your projects.
- **Other clients:** copy the **JSON (HTTP)** text into the client's MCP server list (a project's `.mcp.json`, Cursor's settings, ...).

The port and token are remembered, so a client set up once keeps working after WaveFEM restarts. The agent can only connect while the server is running.

> **Note** Only programs on this computer that know the token can connect. Keep the token private, as you would a password.

Working with an agent

Describe what you want in your own words, for example “build a WR-90 waveguide bend with a 5 mm chamfer and plot S_{11} from 8 to 12 GHz”. Unless you say how the model should be made, the agent asks. It can:

- load a STEP, GDS or DXF file you already have, or one it generates;
- write a CadQuery script for 3D solids such as waveguides, horns and connectors;
- write a gdstk or ezdxf script for planar circuits such as microstrip lines, patches and filters, together with the stack-up. For layouts with curves, asking for ezdxf (DXF) may give a smoother, lighter model.

A script’s top-level numbers appear under **Params** as they do for scripts you load yourself (see Python scripts), so you and the agent can both change them later. When either of you changes some of the parameters, the others keep their current values.

The agent finds faces by their position and direction rather than by clicking, and can render the model with face numbers to check its work. It can also look at what the viewer shows and see the edits you make in the window. Simulations it starts run as usual: the console shows their progress, and the results appear under **Results**.

> **Note** The agent edits the open project directly, so its changes are ordinary edits: **Save** keeps them and closing without saving discards them. Avoid editing the same item as the agent at the same time.

EXAMPLES

Horn Antenna

This example simulates `examples/regular/horn.step` from 3 to 6 GHz.

1. **File** › **Import model file** and choose `horn.step`.
2. Right-click the inner face at the bottom of the waveguide, then **Assign** › **Wave Port**.
3. **Create** › **Material**, **Copy from Library**, choose **PEC**.
4. Click **Body 1** under **Solids**, then **Assign** › **Material** and choose PEC.
5. Click **Setup**, **Edit**, and set `refine_frequency` to `6e9`.
6. **Create** › **Sweep**: 3e9 to 6e9 Hz, 500 steps, adaptive. This gives the S-parameters.
7. **Create** › **Sweep** again: 3e9 to 6e9 Hz, 4 steps, not adaptive, with `save_near_field` and `save_far_field` on. This saves fields at 3, 4, 5 and 6 GHz.
8. **Create** › **Farfield Sweep**: an elevation cut (`sweep_angle` theta, fixed angle 90) and a 3D pattern (`3d_sweep` true, 50 steps).
9. **Simulation** › **Run**.

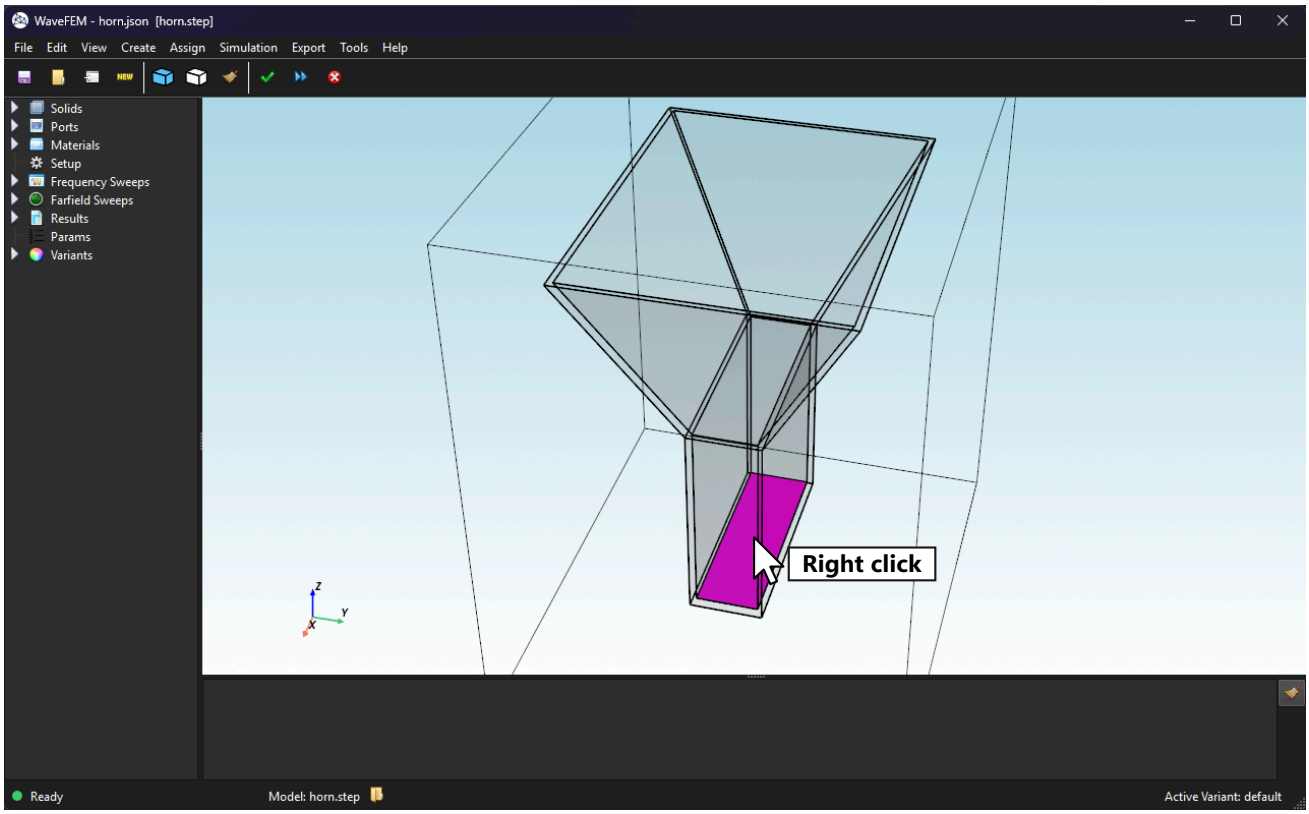


Figure 35 — Wave port at the bottom of the horn

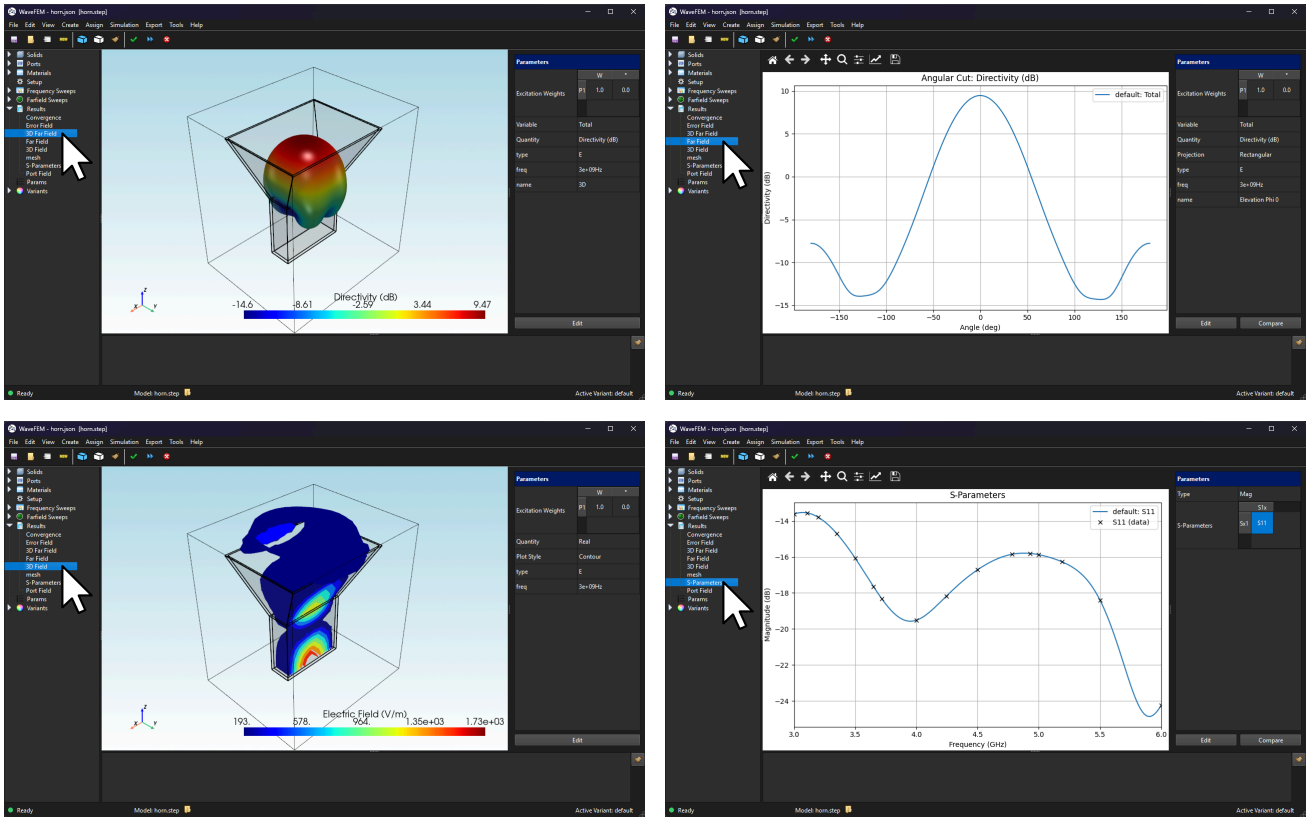


Figure 36 — Horn results: 3D pattern, elevation cut, electric field and S_{11}

EXAMPLES

Microstrip Filter

This example builds a stepped-impedance low-pass filter from `examples/regular/stepped impedance filter.dxf`. The file has two layers: `cond` (the traces, pink) and `cond2` (the ground plane, green).

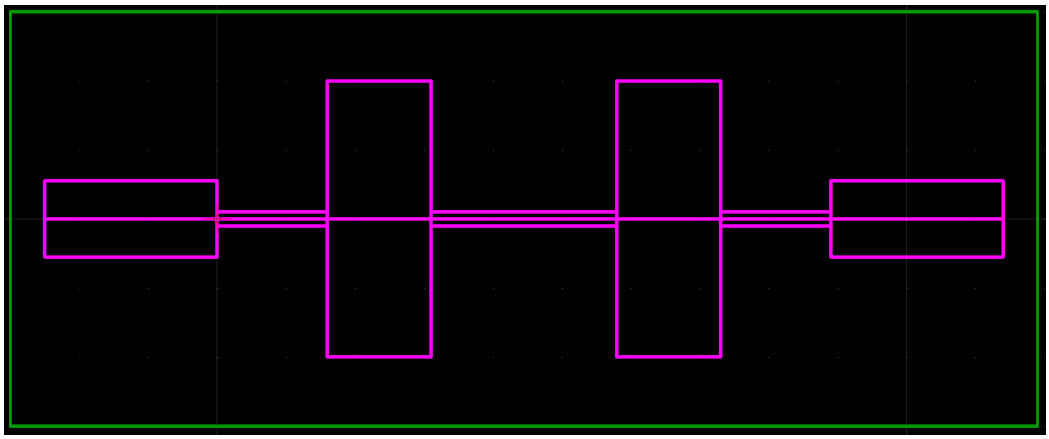


Figure 37 — Filter layout in the DXF file

1. Import the DXF. In the layer window add a dielectric `main`, 0.5 mm thick.
2. Add metal layers `cond` (0.05 mm, on `main`, **Above** and **Intrude Up** ticked) and `cond2` (0.05 mm, on `main`, below). Their names must match the DXF layers. Click **OK**.
3. Create materials `Cond` (PEC) and `RO4003` (`ep` 3.55, `tand` 0). Assign `Cond` to `cond` and `cond2`, and `RO4003` to `main`.
4. Add a lumped port at each end of the line (see Ports).
5. Add an adaptive sweep from 1 Hz to 20 GHz, 500 steps.
6. In **Setup** set `refine_frequency` to `10e9` and `bounding_box_padding` to 4.
7. Run.

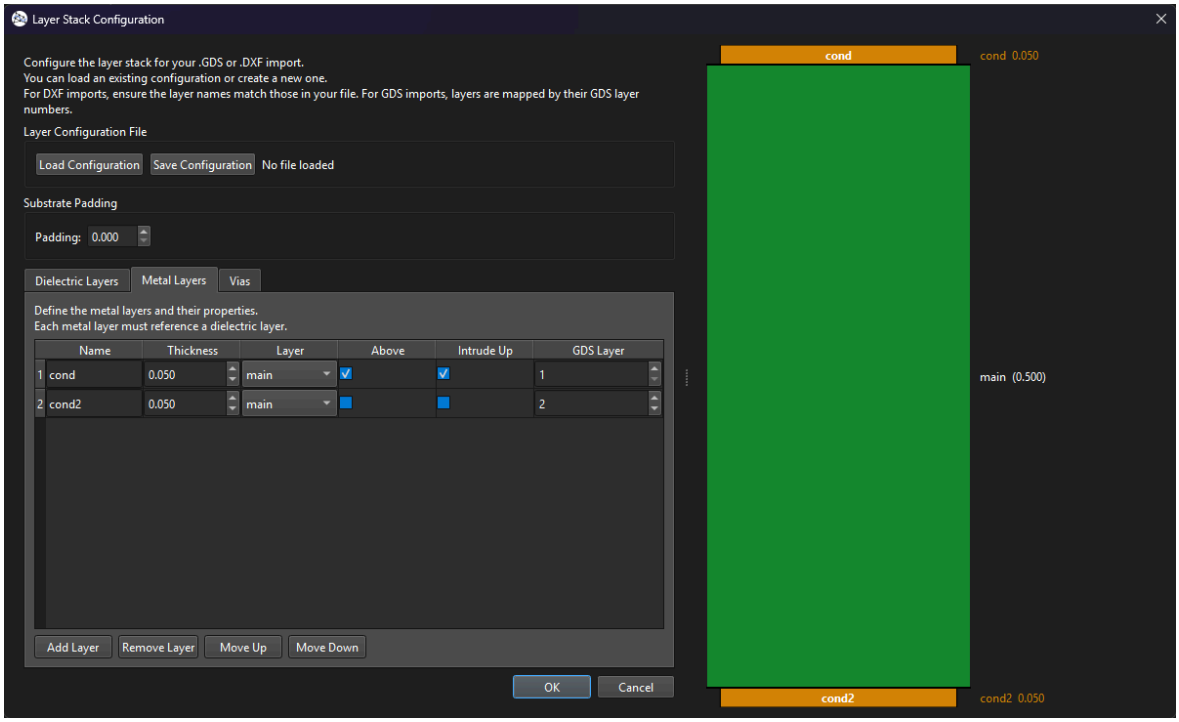


Figure 38 — Stack-up for the filter

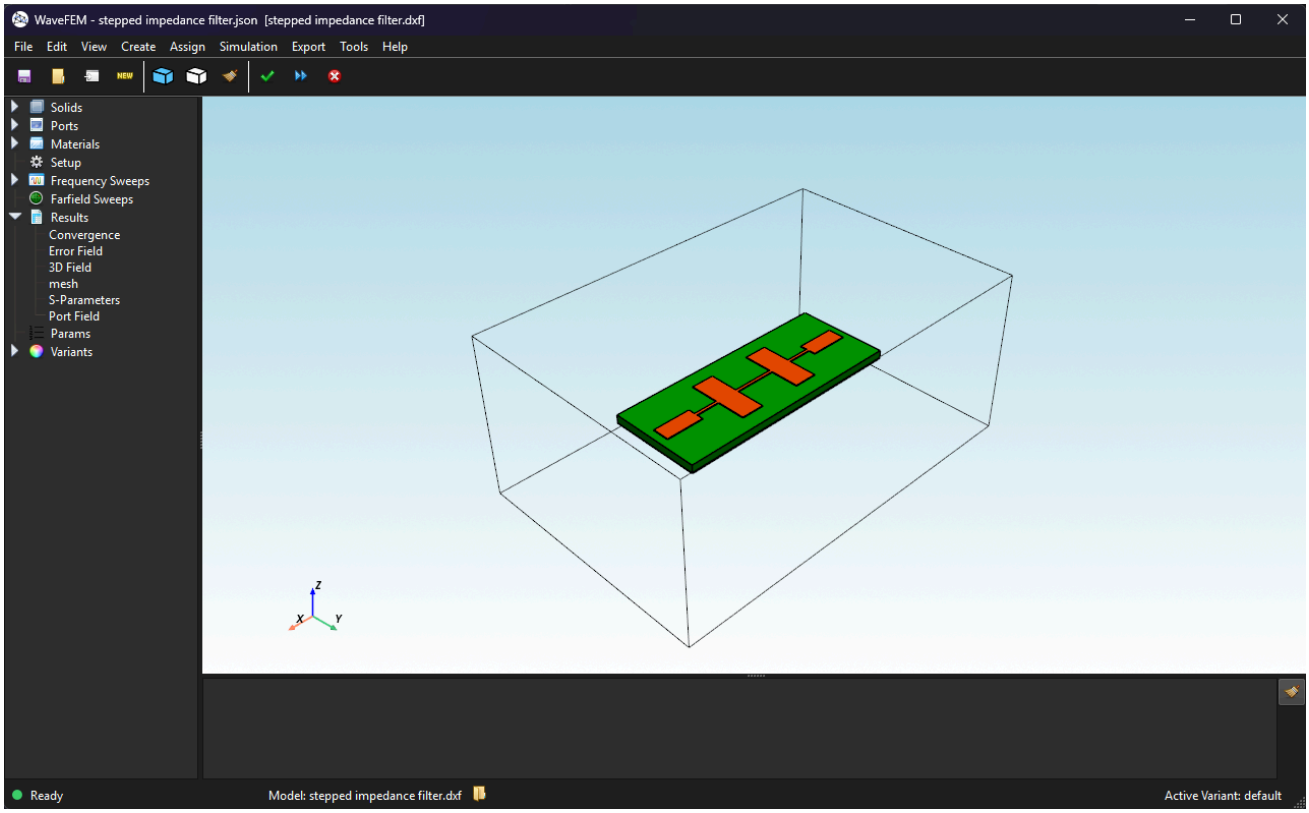


Figure 39 — The filter with materials assigned

EXAMPLES

Scripted Waveguide Bend

This example builds a WR-90 waveguide bend from a CadQuery script, `examples/scripted/scripted waveguide bend.py`, and shows how script parameters make it easy to try design changes.

The script

```python
import cadquery as cq

# Waveguide dimensions
wg_width = 22.86  # Standard WR-90 width (mm)
wg_height = 10.16  # Standard WR-90 height (mm)
wall_thickness = 1.0
straight_length = 50.0  # Length of straight sections
chamfer_length = 17.0  # Length of chamfer at the bend

# Create the outer path for the waveguide (extends beyond inner for caps)
outer_path = (
    cq.Workplane("XY")
    .moveTo(-straight_length - wall_thickness, 0)
    .lineTo(0, 0)
    .lineTo(0, straight_length + wall_thickness)
)

# Create the inner path for the hollow interior
inner_path = (
    cq.Workplane("XY")
    .moveTo(-straight_length, 0)
    .lineTo(0, 0)
    .lineTo(0, straight_length)
)

# Create outer cross-section profile
outer_profile = cq.Workplane("YZ").rect(
    wg_width + 2 * wall_thickness, wg_height + 2 * wall_thickness
)

# Create inner cross-section profile
inner_profile = cq.Workplane("YZ").rect(wg_width, wg_height)

# Create the outer and inner shapes by sweeping profiles along their respective paths
outer_shape = (
    outer_profile.sweep(outer_path).edges(">(1, -1, 0)").chamfer(chamfer_length)
)
inner_shape = (
    inner_profile.sweep(inner_path).edges(">(1, -1, 0)").chamfer(chamfer_length)
)

# Create hollow waveguide with caps by subtracting inner from outer
result = outer_shape.cut(inner_shape)

out_file = "waveguide_bend.step"
result.export(out_file)
```

How it works

1. **Parameters.** The variables at the top (`wg_width`, `wg_height`, `wall_thickness`, `straight_length`, `chamfer_length`) are plain numbers. WaveFEM lists every top-level assignment of a literal value under **Params**, so these can be changed without editing the file.
2. **Paths.** Two L-shaped paths trace the centre line of the guide: one for the outside of the walls and one for the hollow inside. The outer path is longer by `wall_thickness` at each end, which leaves a wall across each opening.
3. **Profiles.** Two rectangles on the YZ plane give the cross-sections: the inside is the standard 22.86 × 10.16 mm WR-90 aperture, the outside adds the wall thickness all round.
4. **Sweep and chamfer.** Each profile is swept along its path into a solid L-shaped bar, and the outer corner of the bend is cut off by `chamfer_length` to give a mitred bend.
5. **Cut.** Subtracting the inner solid from the outer one leaves the metal body: a hollow guide closed at both ends. The two end caps are the faces used as wave ports.
6. **Export.** The script must set `out_file` and write the model to it. WaveFEM redirects this to a generated file next to the script (here `waveguide_bend[auto_generated].step`) and loads that.

> **Note** Build models from solids, as here: sweep, extrude or revolve closed profiles and combine them with `cut` and `union`. Do not use `shell()` to hollow a part. Shelling renumbers faces whenever a parameter changes, so ports and other assignments end up on the wrong faces, and it sometimes leaves a solid that is not watertight, which cannot be meshed.

Debug a new script in a Python editor first. Errors appear in a dialog when the model loads, and stay in the console (see Python scripts).

Simulating it

1. **File** › **Import model file** and choose the `.py` file. Its parameters appear under **Params**.
2. Right-click the end cap of one arm and **Assign** › **Wave Port**. Repeat on the other arm.
3. Create a PEC material and assign it to the body.
4. In **Setup** set `refine_frequency` to `8e9` and `bounding_box_padding` to `10`.
5. Add a sweep from 7 to 8 GHz and run.

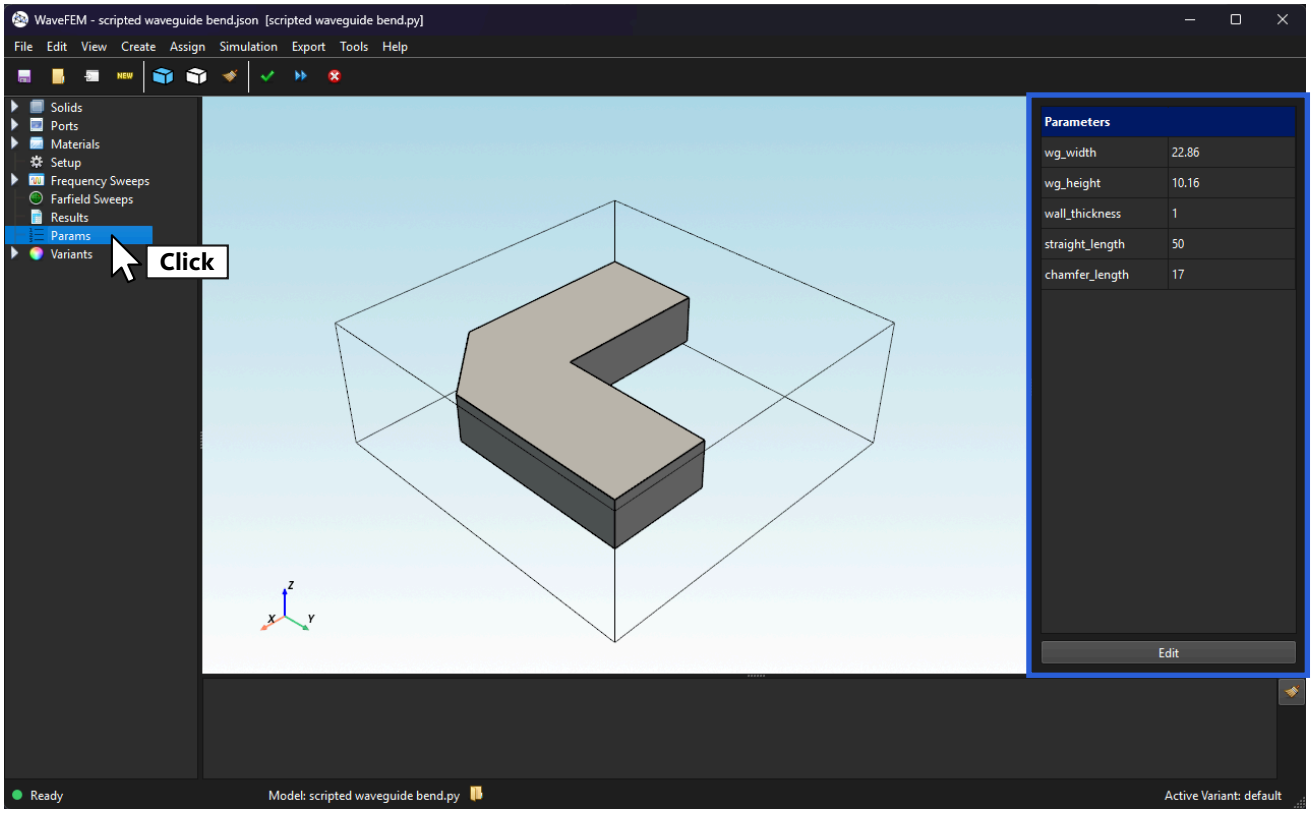


Figure 40 — The script's parameters in WaveFEM

Trying a design change

1. Select the `default` variant under **Variants** and click **Copy**.
2. Activate the copy, click **Params** and change `chamfer_length`. The model rebuilds straight away; the ports stay on the same end faces because the face order does not change.
3. Run the new variant, then open **S-Parameters** and click **Compare** to plot both variants together. A good chamfer gives the lowest S_{11} across the band.

APPENDIX

References

[1] D. B. Davidson, "An evaluation of mixed-order versus full-order vector finite elements," *IEEE Transactions on Antennas and Propagation*, vol. 51, no. 9, 2003.

[2] B. Gustavsen, "Rational approximation of frequency domain responses by vector fitting," *IEEE Transactions on Power Delivery*, vol. 14, no. 3, pp. 1052–1061, 1999.